



软件架构设计

主讲：李伟刚

liweigang@nwpu.edu.cn

西北工业大学软件学院



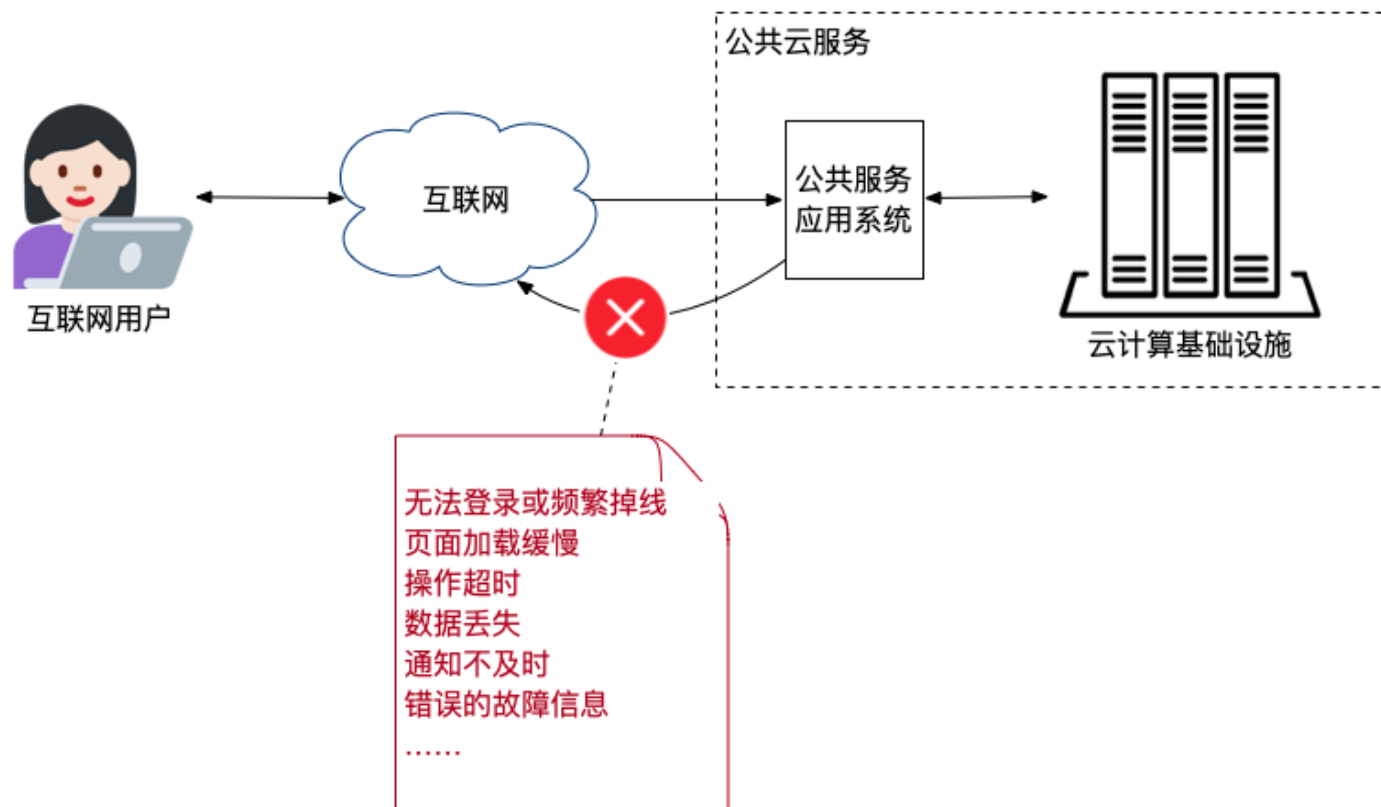
第十三章 大数据技术应用架构实践

主要内容

- 1. 税务系统健康监测架构设计案例
- 2. 医疗系统数据同步架构设计案例



1. 税务系统健康监测架构设计案例



可靠性，是公共服务最本质与核心条件之一，直接关系到公众对服务的信任基础。

- 具体有哪些资源需要进行状态监控？



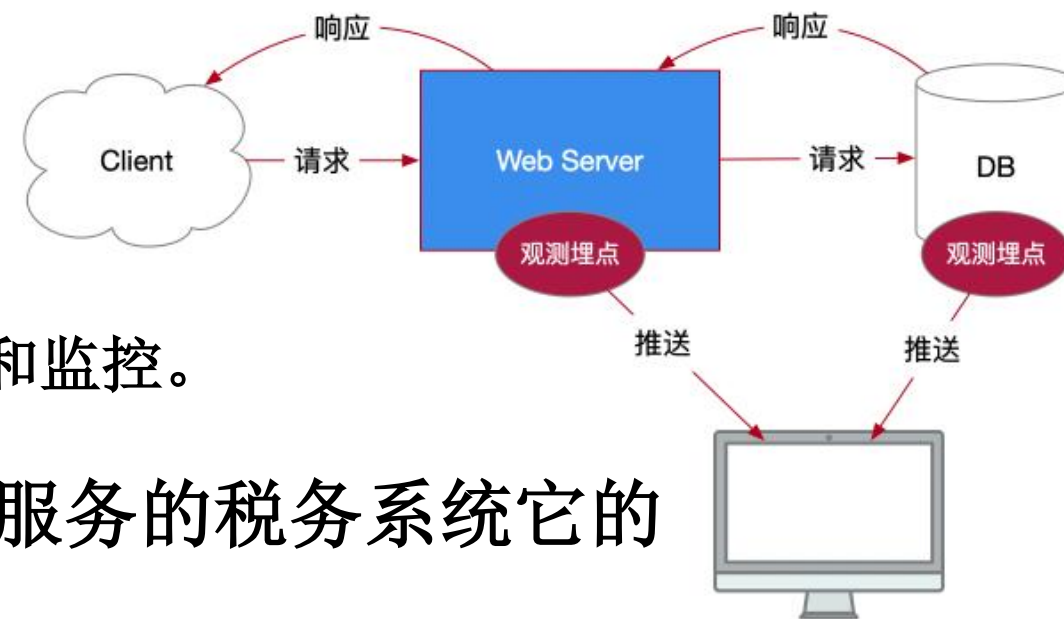
1. 税务系统健康监测架构设计案例

- 小型网站的基础监控项

- 服务器监控：CPU、内存、磁盘、网络流量等。
- Web服务监控：服务是否在线、响应时间、并发连接数等。
- 数据库监控：连接数、查询响应时间、磁盘使用情况、缓存命中率等。

- 基础监控项的工具/手段

- 系统命令：top、vmstat、iostat等。
- MySQL状态查询例如SHOW STATUS
- Nginx日志采集、解析和分析。
- Prometheus和Grafana基础指标采集和监控。



- 对于一个数据中心，例如面向公共服务的税务系统它的应用系统有多复杂呢？

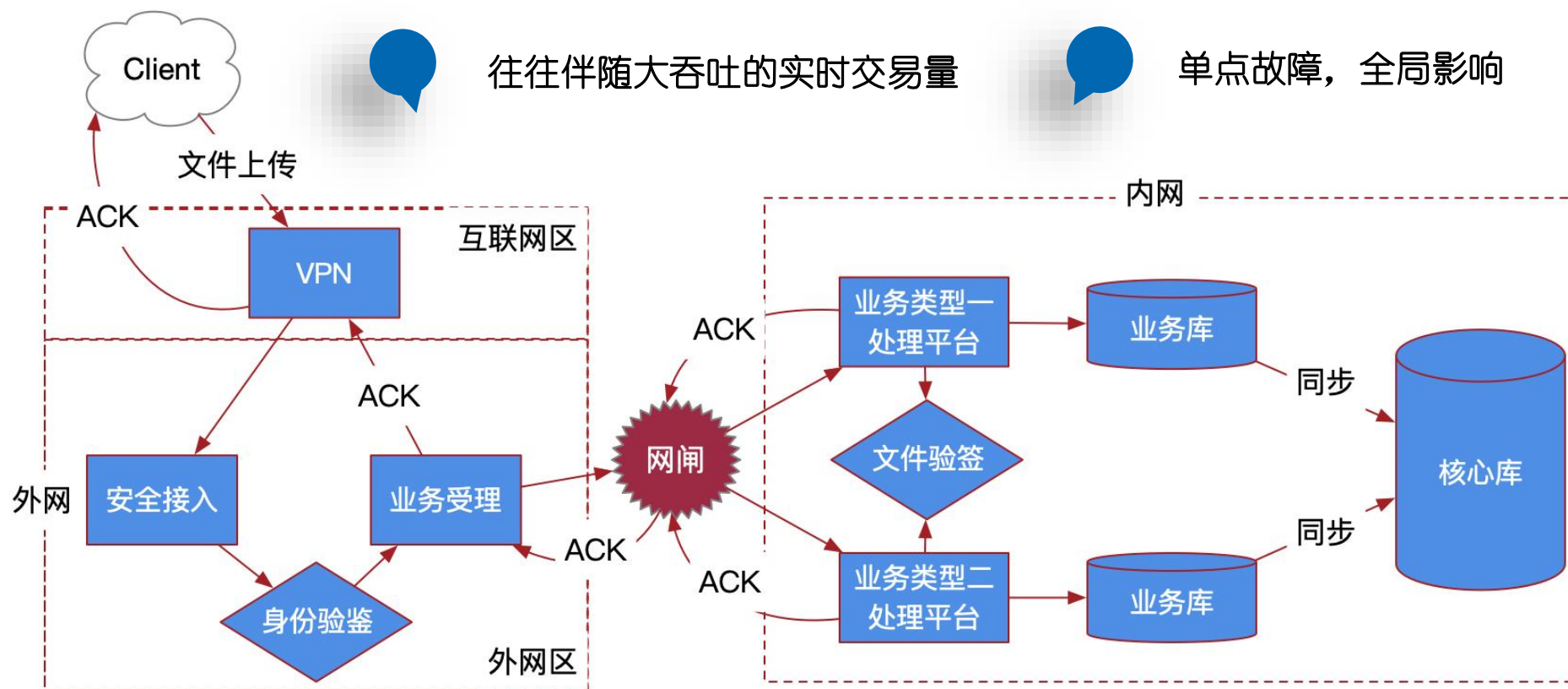


1. 税务系统健康监测架构设计案例

- 税务系统应用场景

- 复杂的网络体系、复杂的业务系统协同

- 如此复杂的数据中心的应用系统都有哪些监控手段呢？



1. 税务系统健康监测架构设计案例

● 数据中心基础设施监控

- 操作系统、网络设备、存储
- Web服务器、消息队列
- 数据库、虚拟化、云平台
- 安全服务、物理设施

● 如此多的监控手段、监测指标，是否能快速捕获到应用故障的根源？

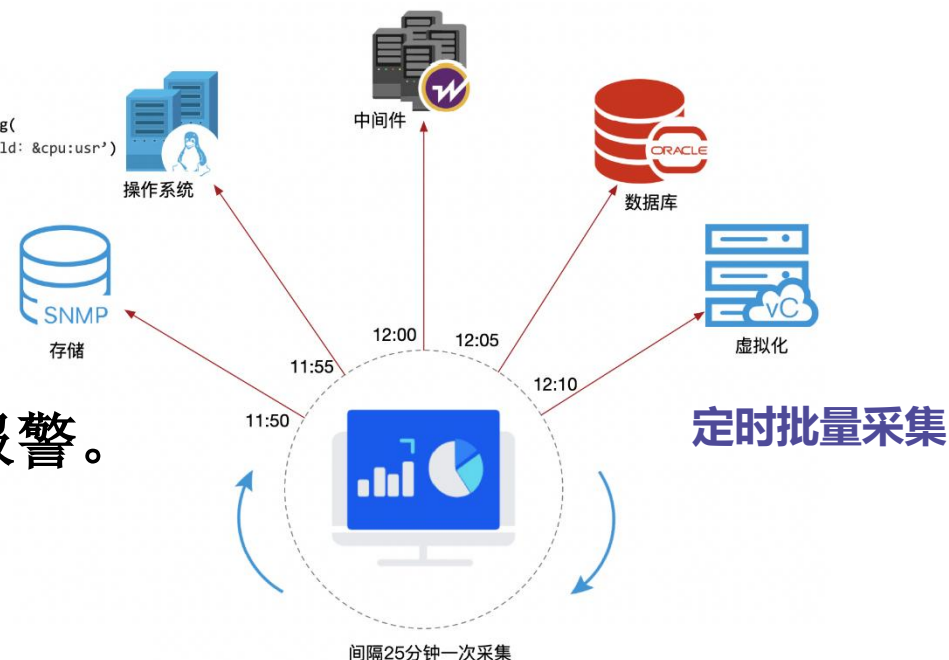


1. 税务系统健康监测架构设计案例

● 复杂的云服务环境运维的难点

- 表象：数据库连接数高。
 - 实际：上游服务超时导致。
- 表象：整体流量暴增。
 - 实际：某个**API**端点响应慢导致重试次数增多。
- 表象：大数据存储对内存占用率极高导致报警。
 - 实际：系统正常运行。
- 表象：服务的计算资源消耗巨大。
 - 实际：第三方**API**限流或者响应缓慢。
- 表象：前端无法完成用户请求，应用服务看似故障。
 - 实际：消息队列任务积压导致资源耗尽。

```
if [cpu:usr > 93]; then SendAlarmMsg(  
    'The CPU usage exceeds the threshold: &cpu:usr')  
fi
```



往往症状表
象与实际病
因不一致

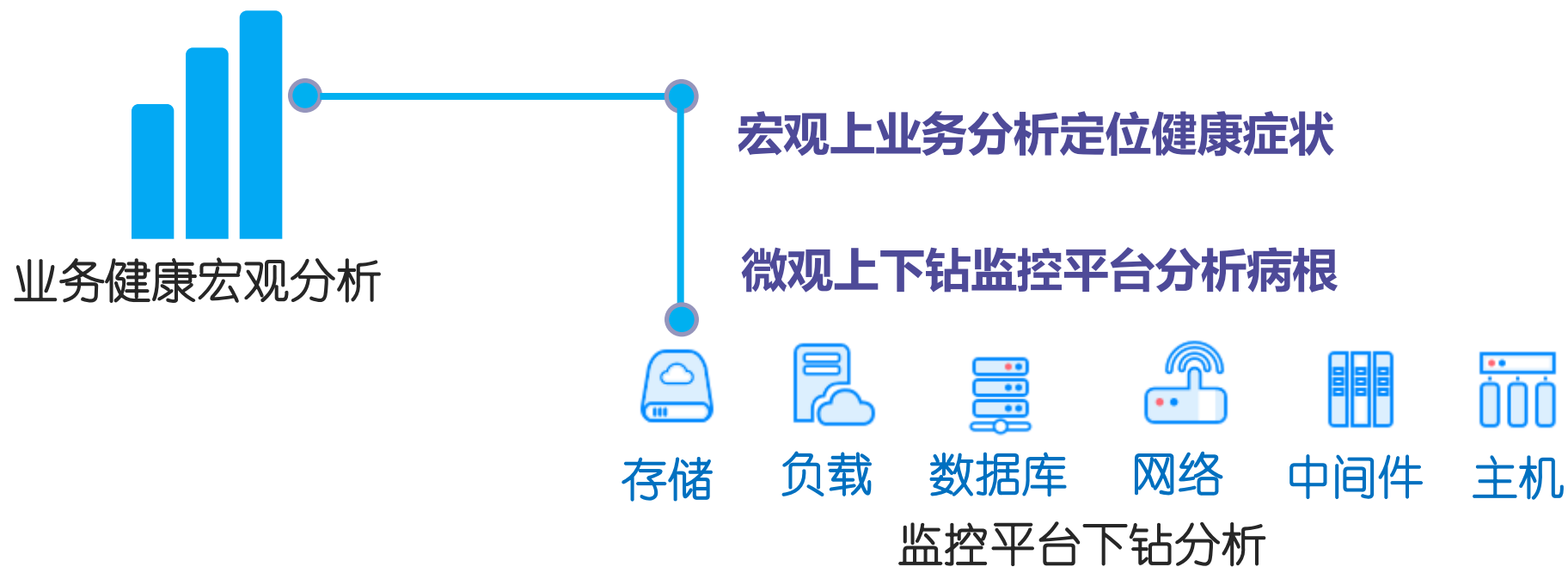


西北工业大学

NORTHWESTERN POLYTECHNICAL UNIVERSITY

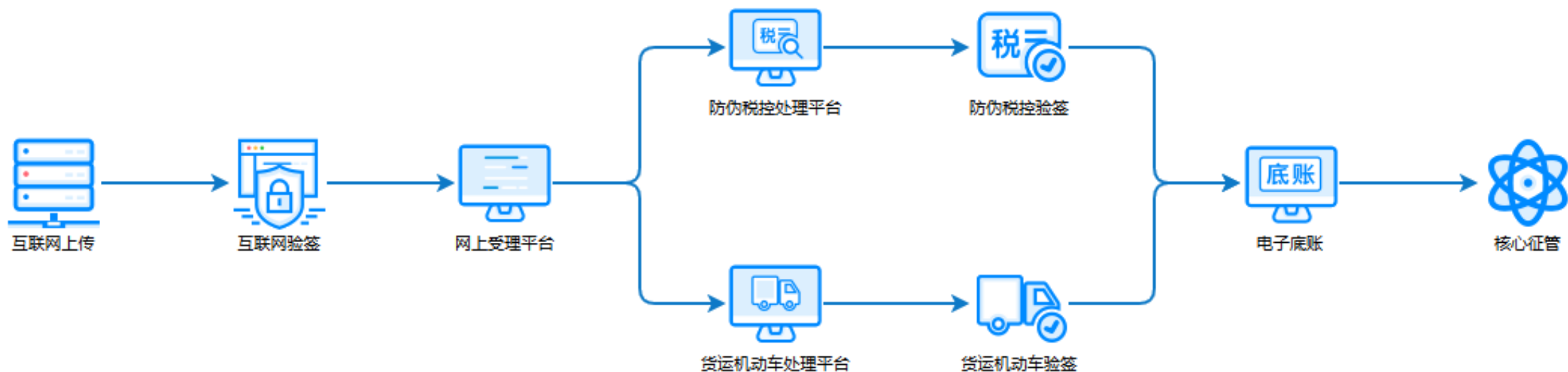
1. 税务系统健康监测架构设计案例

- 基于“业务健康症状”分析的监控架构



1. 税务系统健康监测架构设计案例

- “业务健康症状”分析的核心——业务数据流的实时轨迹观测
 - 健康监测架构的基础就是业务服务间紧密的流程关系。



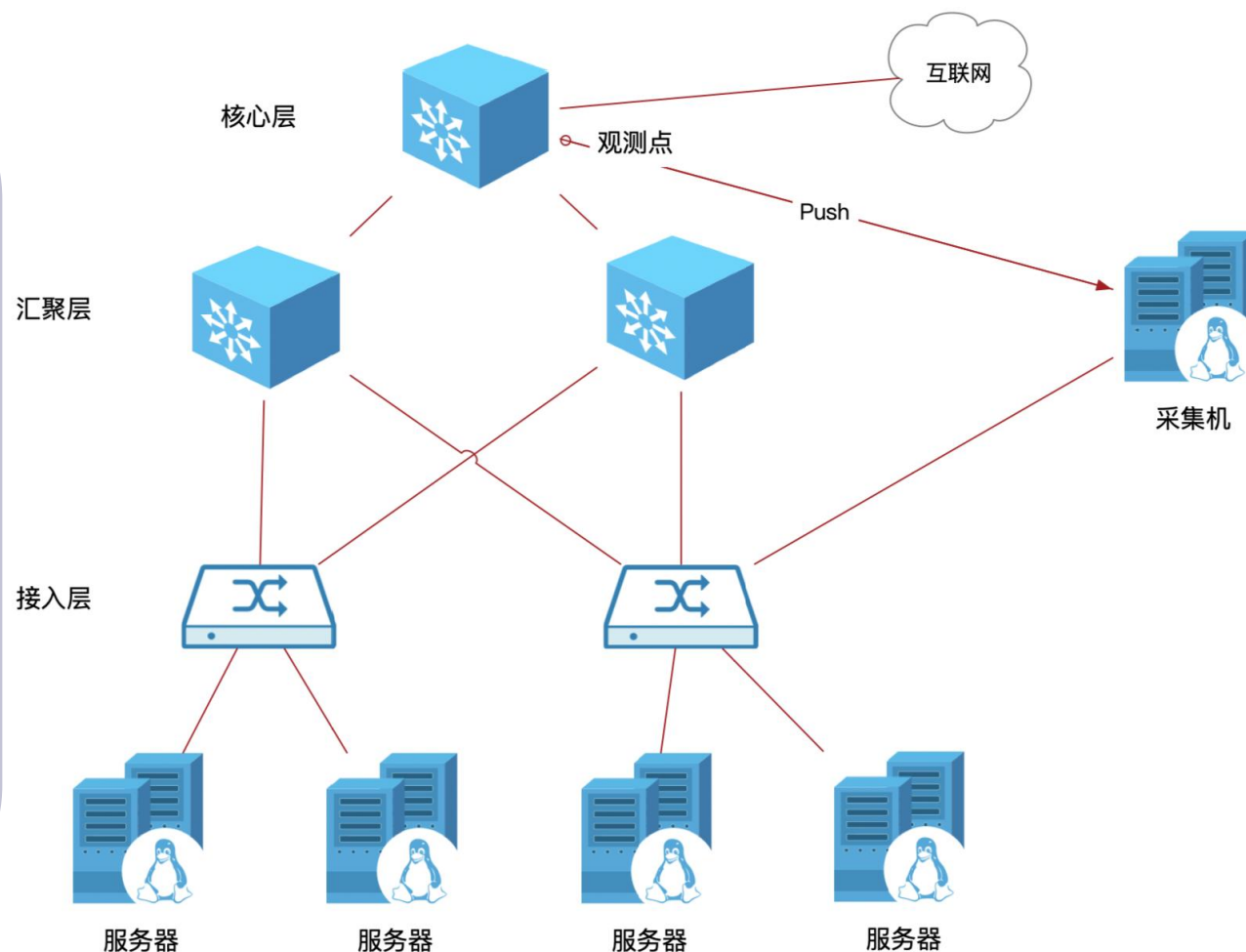
对流经各计算服务的业务数据流的采样、分析，实时监控



1. 税务系统健康监测架构设计案例

● 业务数据流实时轨迹观测的数据采集手段

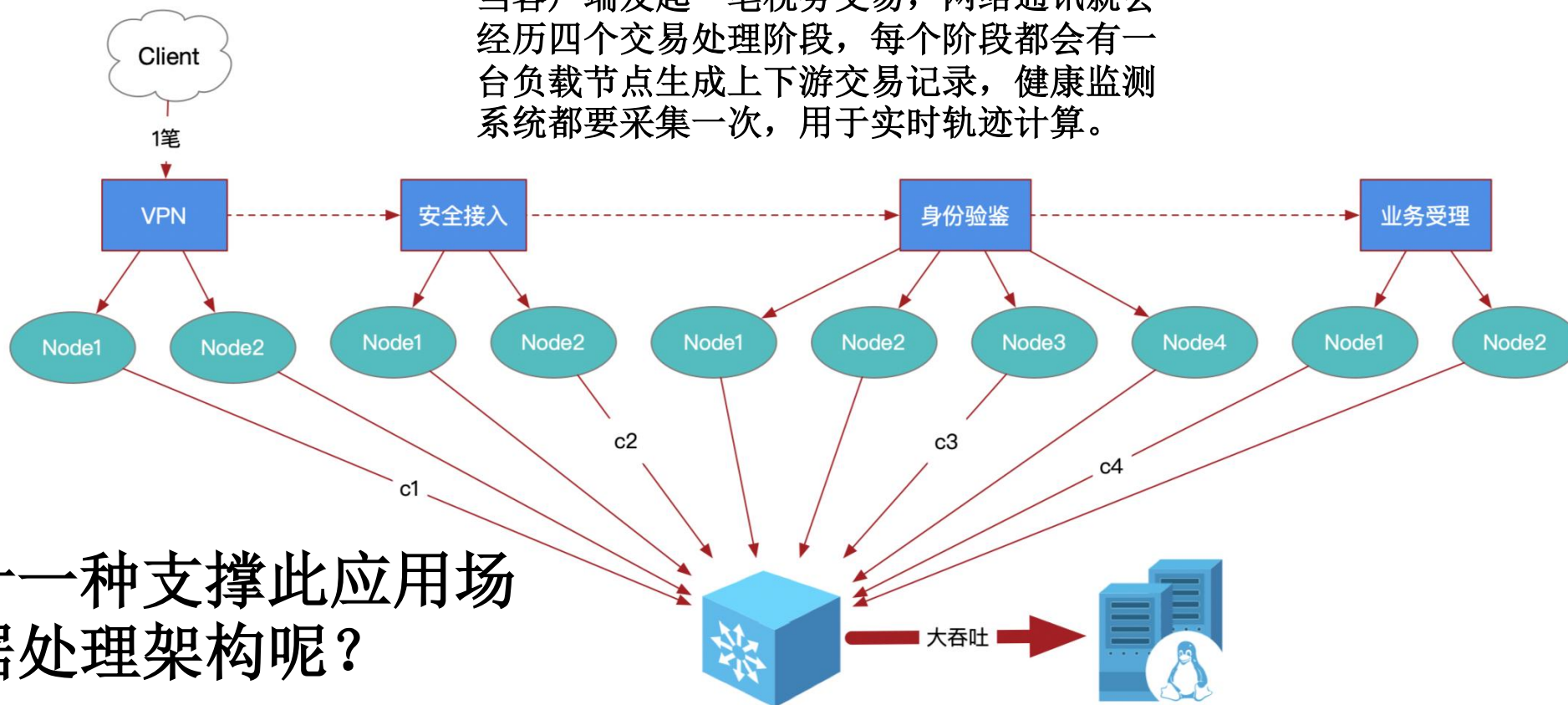
- 业务链路所涉及的服务节点的网络请求响应数据都可以捕获。
- 流经采集服务器的时间作为标准参考时间戳。
- 核心交换机镜像数据**Push**不会带来额外负载。
- 数据包的业务信息重组，关联业务响应内容，计算出业务轨迹的各项健康指标，例如业务每个环节的性能延时。



1. 税务系统健康监测架构设计案例

- 省级税务征收一天入库的数据量过百GB
 - 健康监测架构对入库数据形成了全业务链路的多点复制、单点汇聚的扇形结构。

当客户端发起一笔税务交易，网络通讯就会经历四个交易处理阶段，每个阶段都会有一台负载节点生成上下游交易记录，健康监测系统都要采集一次，用于实时轨迹计算。



- 如何设计一种支撑此应用场景的数据处理架构呢？

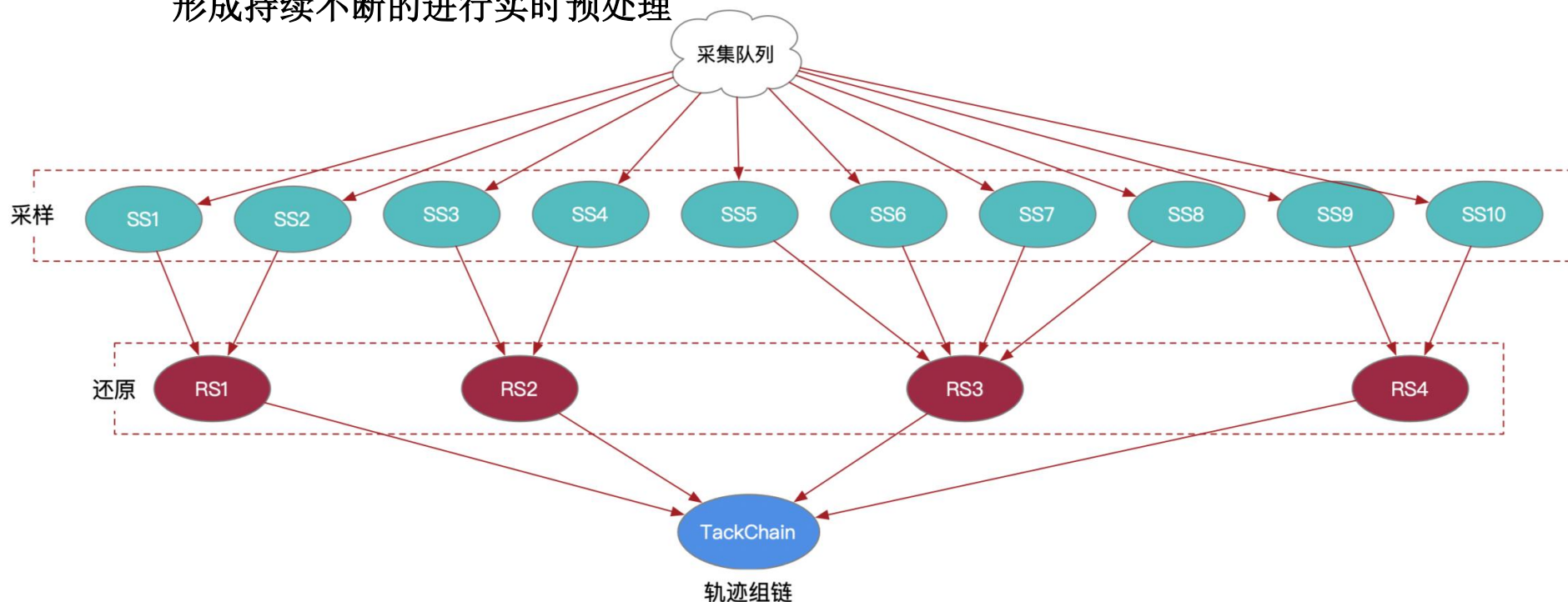


1. 税务系统健康监测架构设计案例

- 保证海量大吞吐数据的实时预处理架构

- 必须借助分布式计算实现数据分流处理再汇聚。

通过采集队列的进行扇形数据分流，整个分布式处理架构形成了有向无环图 **DAG (Directed Acyclic Graph)**，每个并行计算分片均衡整体的吞吐量，形成持续不断的进行实时预处理



1. 税务系统健康监测架构设计案例

● 进一步理解流式并行计算

○ Apache Storm 进行分词计数的例子

1、实例化一个TopologyBuilder对象来定义应用程序拓扑结构

```
TopologyBuilder builder = new TopologyBuilder();
```

2、添加一个Spout并产生句子流。该Spout被命名为sentence-spout，并被指定并行度为2，这意味着有2个线程将跨集群执行该Spout

```
builder.setSpout( id: "spout", new RandomSentenceSpout(), parallelismHint: 2);
```

3、需要一个Bolt来使用该流并将其转换成单词流。该Bolt被称作分流器（Splitter），且并行度为4。因为没有要求如何消费句子，所以使用随机分组来均匀地在所有4个任务上分布处理负载

```
builder.setBolt( id: "split", new SplitSentenceJVM(), parallelismHint: 4)
    .shuffleGrouping("spout");
```

4、最后一个Bolt使用单词流并产生所需单词计数的流。且并行度为3。应使用字段分组，以确保只有一个任务负责确定任何特定单词的总数：

```
builder.setBolt( id: "count", new WordCountBolt(), parallelismHint: 3)
    .fieldsGrouping( componentId: "split", new Fields("word"));
```

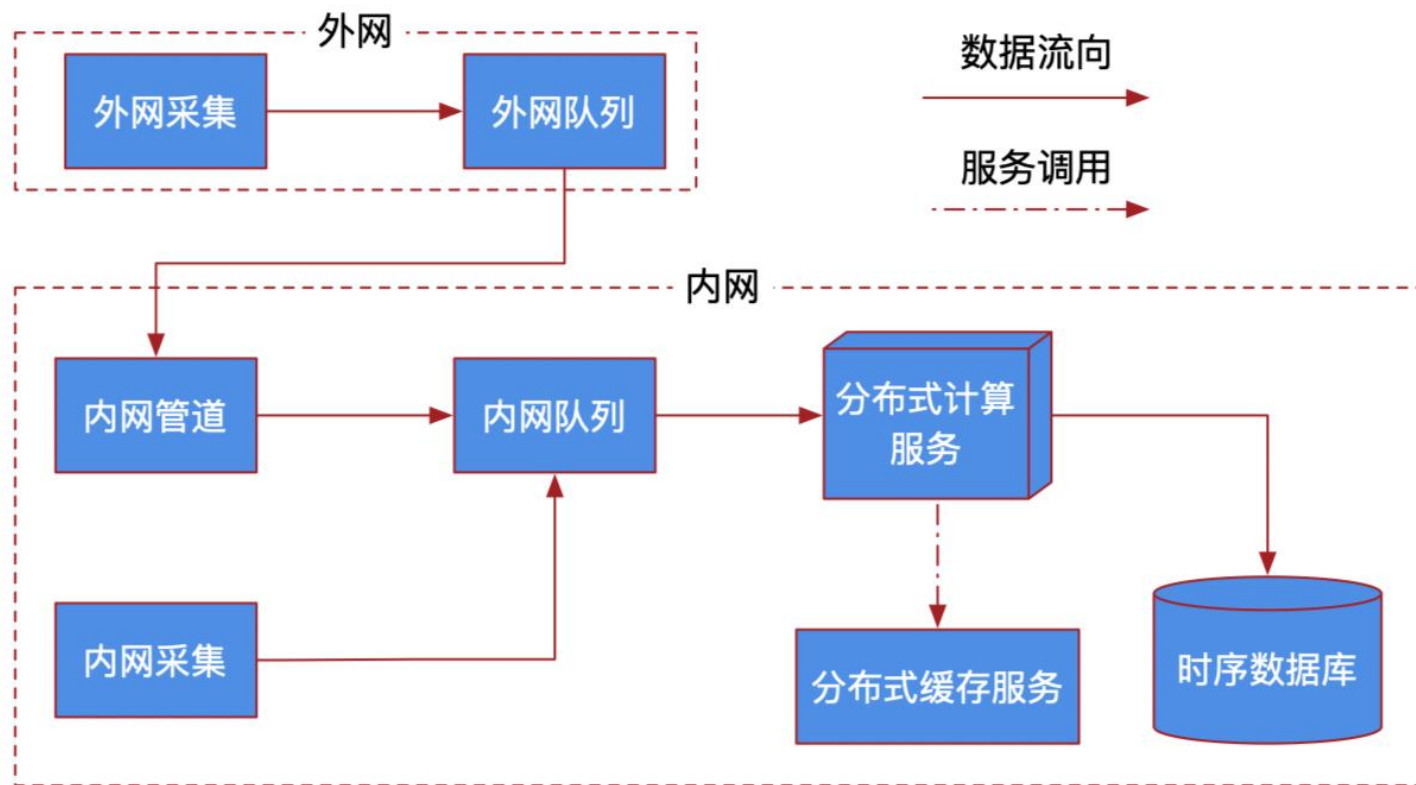
1. 2个数据源不断生成句子并随机地推送给分词器。
2. 4个分词器并行地将句子分解为多个单词，通过计算单词HASH值的方法，推送给相同的计数器。
3. 3个计数器分担统计任务，为每个单词建立一个K/V统计池，不断更新单词出现的频次。



1. 税务系统健康监测架构设计案例

- 税务系统健康监测数据处理部分的软件架构概览

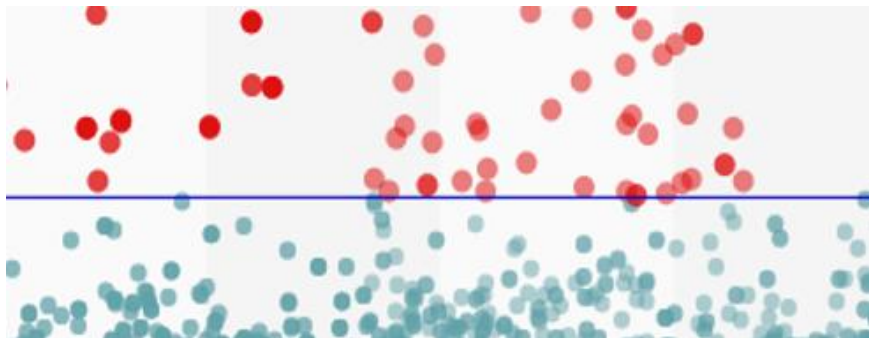
- 流式处理包括了：采集、队列、管道、缓存、处理、存储等分布式组成部分。



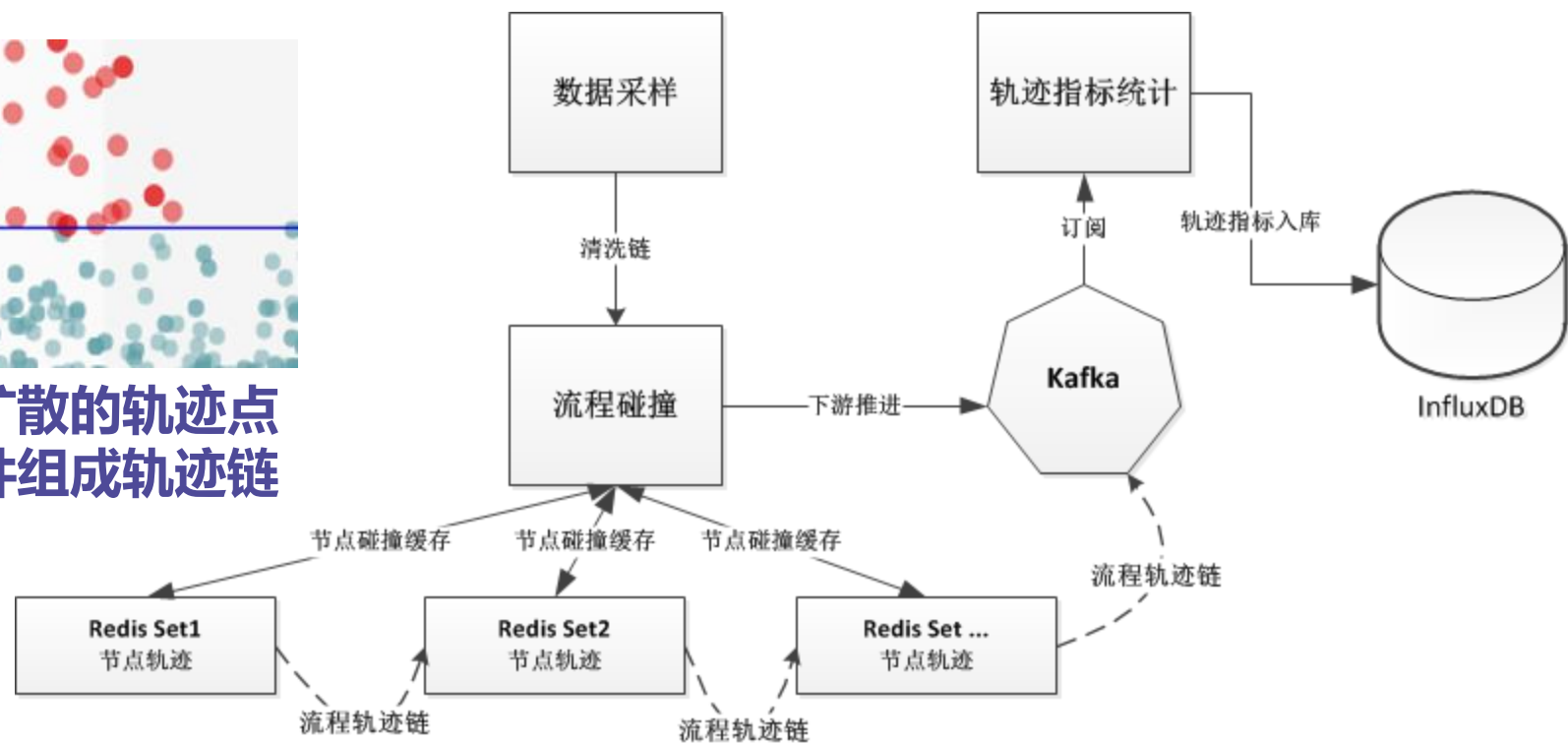
1. 税务系统健康监测架构设计案例

● 健康度监测的轨迹跟踪机制

- 轨迹跟踪的核心是流程轨迹链的碰撞连接过程，借助**Redis**缓冲，高速碰撞，步进式组链，完成流程轨迹跟踪。



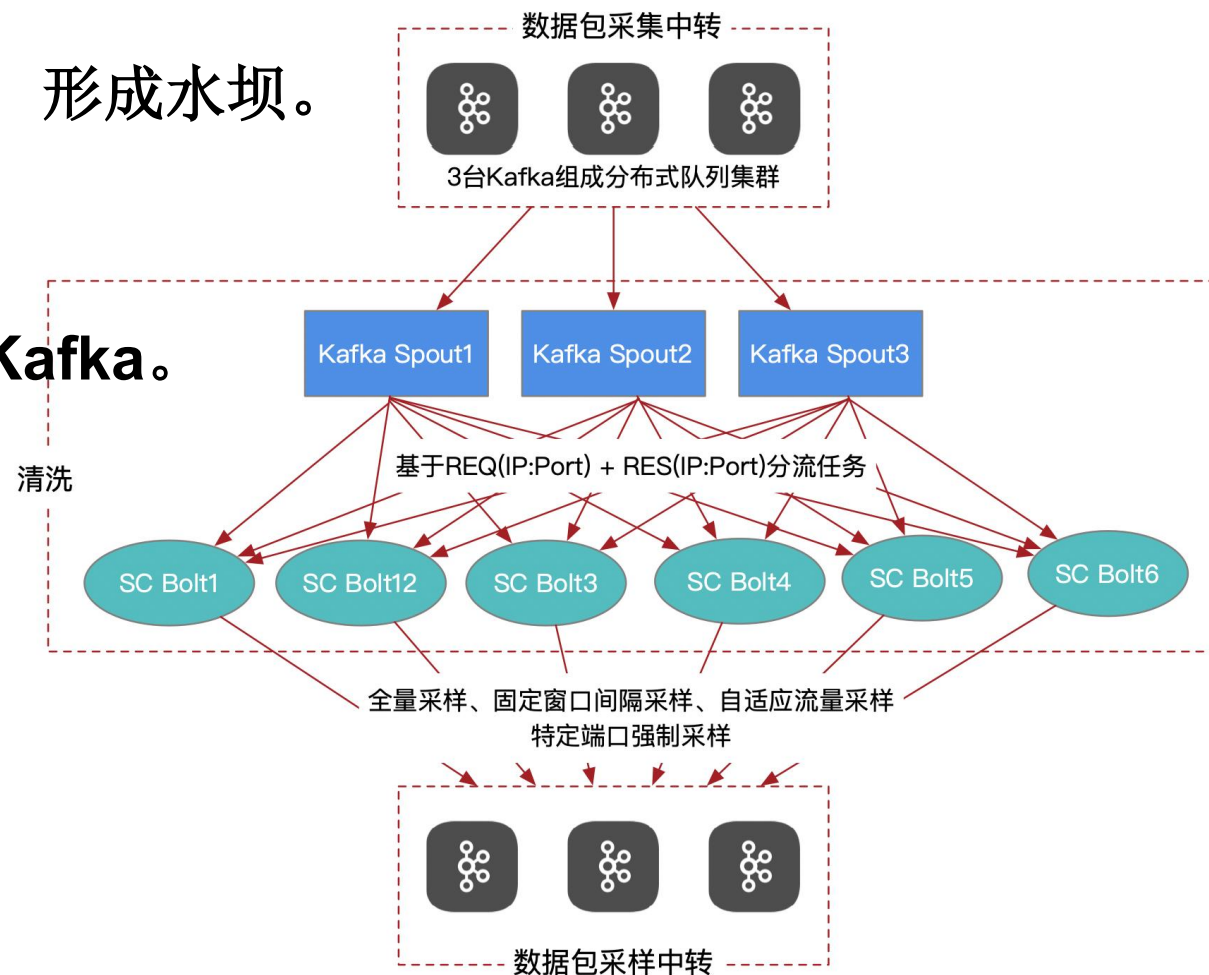
税务业务交易事件如同扩散的轨迹点
通过碰撞将相同交易事件组成轨迹链



1. 税务系统健康监测架构设计案例

● Kafka集群在流式架构中的作用

- **Kafka**接收交换机采集并封装的事件，形成水坝。
- 通过**REQ & RES**分类形成事件分流。
- 每条流水线上进行事件采样。
- 采样后(数据清洗)的结果再次汇聚到**Kafka**。
- 清洗后的数据用于轨迹链碰撞。



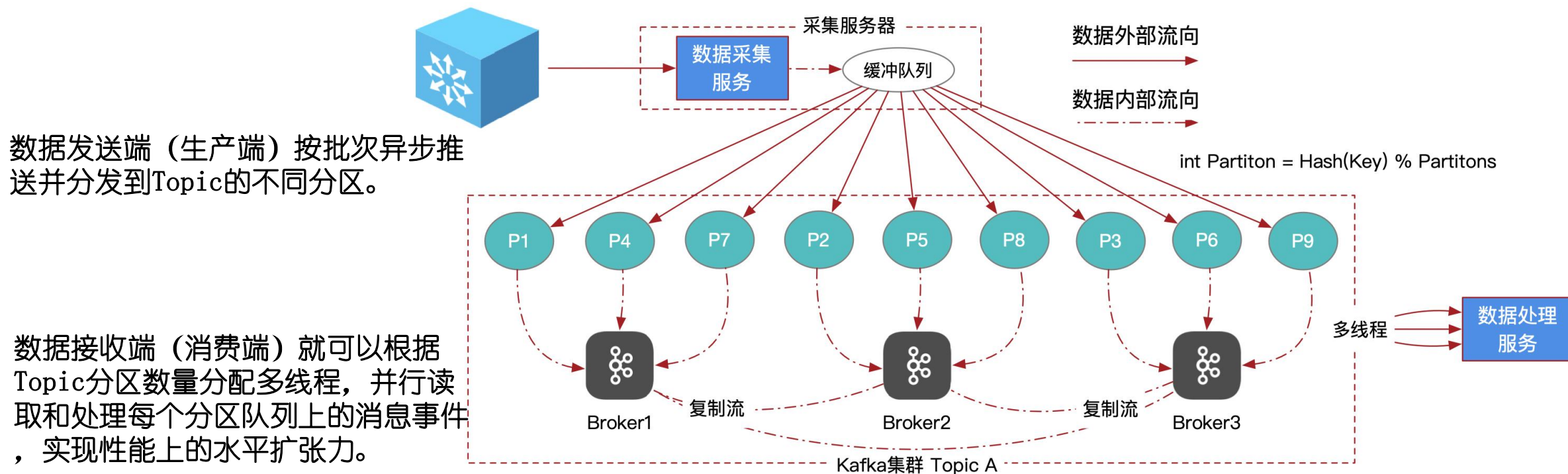
● Kafak在实时流计算架构中的作用举足轻重。让我们一窥Kafka的分布式架构设计！



1. 税务系统健康监测架构设计案例

● Kafka集群的分布式架构

- 集群由多台Broker组成，Broker被划分为多个分区用于存储数据。
- 数据流写入Topic，Topic将数据分布在不同Broker的分区上。
- 写入分区的数据以一主多从的模式复制到其他Broker的分区上实现高可靠。

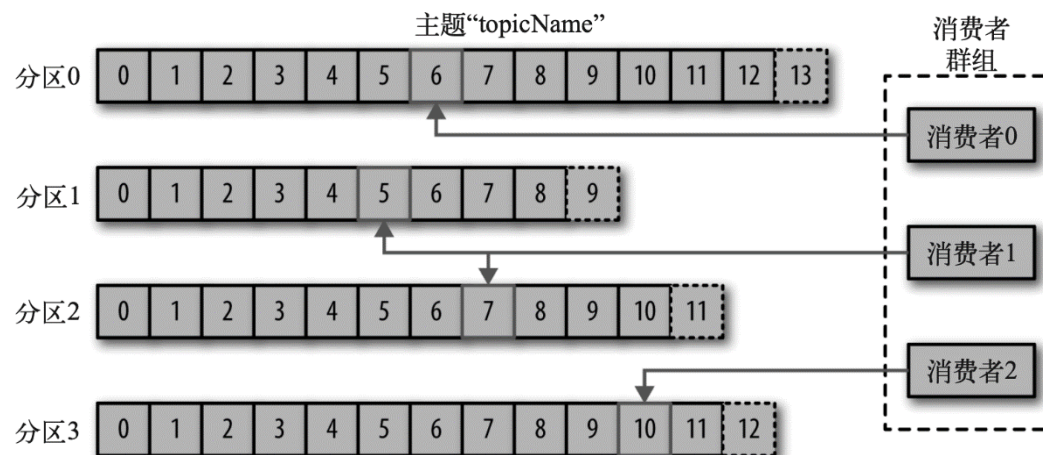
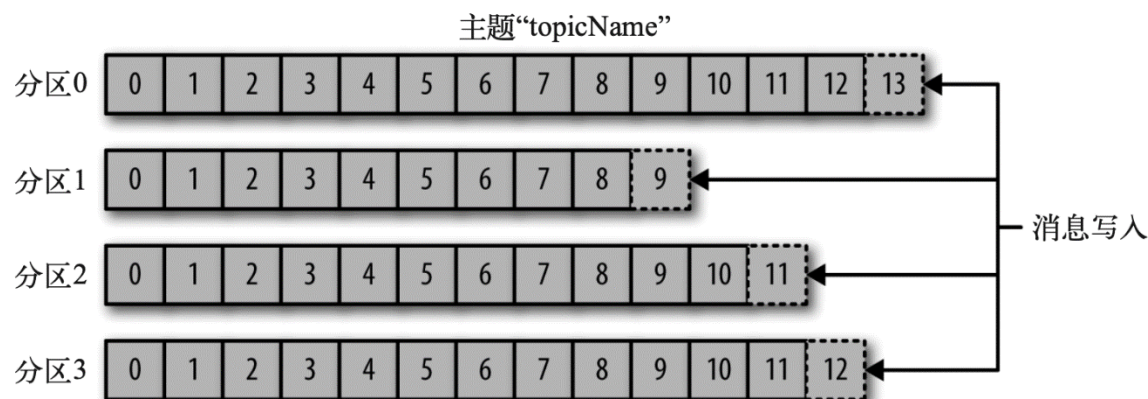


1. 税务系统健康监测架构设计案例

● Kafka的生产者与消费者

- 在分区内数据是队列的形式排序。
- 数据直接写入磁盘日志文件。
- 读取通过**offset**寻址并提交。

生产者写入数据

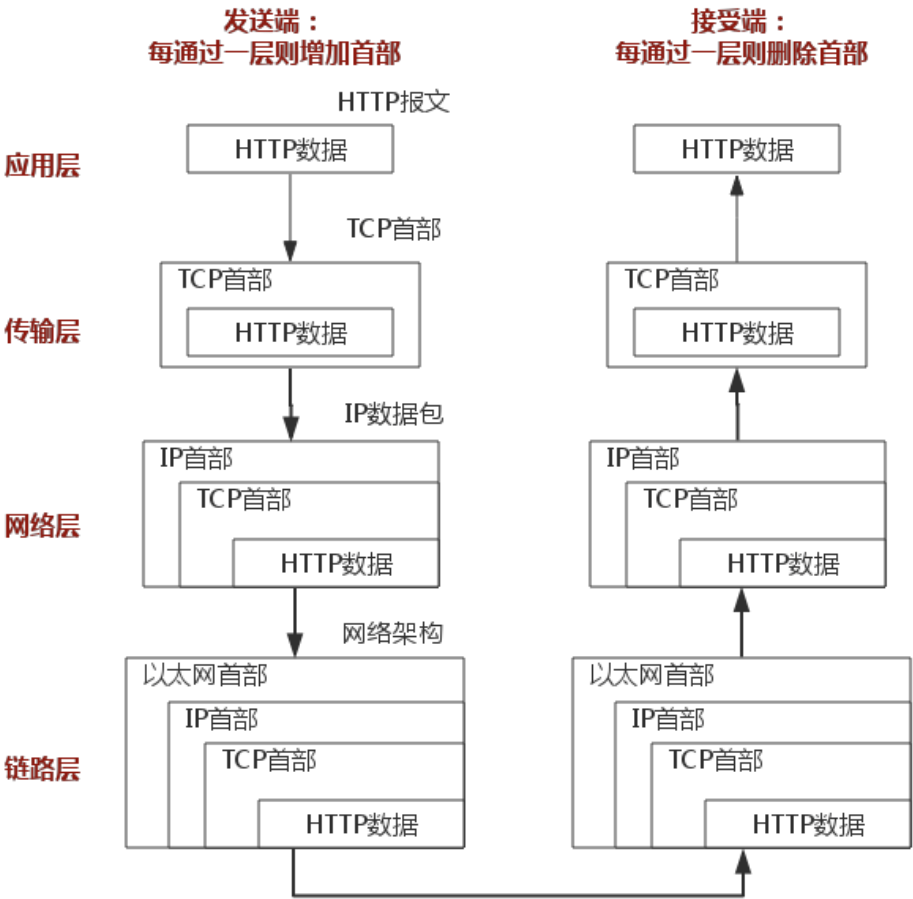


消费者读取数据



1. 税务系统健康监测架构设计案例

● 税务系统网络数据包采集样例



【第1个报文开始】

timeStamp:1513237800000652948
<IP数据报分析>,版本号:4,首部长度:20,TOS:0,总长度:646,标识:10049,df:1,mf:0,偏移量:0
间:125,协议号:6,首部校验和:55997,源IP地址:71.12.91.30,目的IP地址:71.17.16.56
<此报文是TCP报文>源端口号:62954,目的端口号:7001,序号:3516245166,确认号:609598860,
5,URG:0,ACK:1,PSH:1,RST:0,SYN:0,FIN:0,窗口大小:257,检验和:1454,紧急指针:0

内容:POST /SjbWsywServlet/SkpDo HTTP/1.1
Content-Type: application/x-www-form-urlencoded; charset=GB2312
User-Agent: Jakarta Commons-HttpClient/3.1
Host: 10.48.4.30:7001
Content-Length: 414

```
<?xml version="1.0" encoding="gbk"?><root  
id=""><digitalEnvelope>0</digitalEnvelope><zip>0</zip><context><?xml version="1.0"  
encoding="gbk"?><business id="FPMXJG" comment="发票明细结果"><body count="1"  
skph="499099928874" nsrsbh="91152501816090360Y"><group xh="1"><data name="fplx_dm"  
value="007"/></group></body></business></context></root>
```

通过纳税人识别号实施轨迹跟踪



西北工业大学

NORTHWESTERN POLYTECHNICAL UNIVERSITY

1. 税务系统健康监测架构设计案例

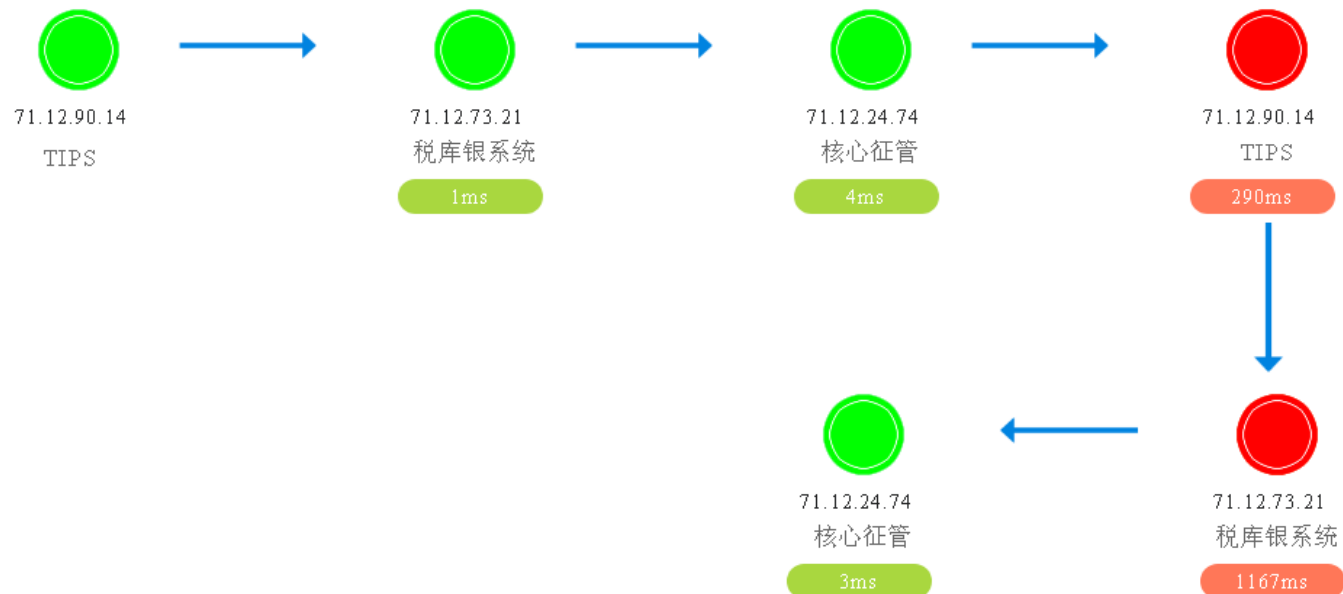
- 税务系统健康监测轨迹跟踪的性能指标样例
 - 业务轨迹跟踪性能指标显示了前后轨迹点的性能耗时。

业务轨迹一览

查询日期：2018-04-25 20:50:54 纳税人识别号： 92150821MA0NJJ863Q 发票号码：

我们会发现TIPS和税库银在二次业务调用流程中性能延时极大，该过程中经历了税务系统与银行系统的支付提交。

因此通过这种定位，就可以进一步下钻到税库银和TIPS的运维系统，采集这段时间内与支付提交服务相关的各项平台监控指标，发出预警消息，提醒综合运维系统或人员聚焦这段时间的异常问题分析。



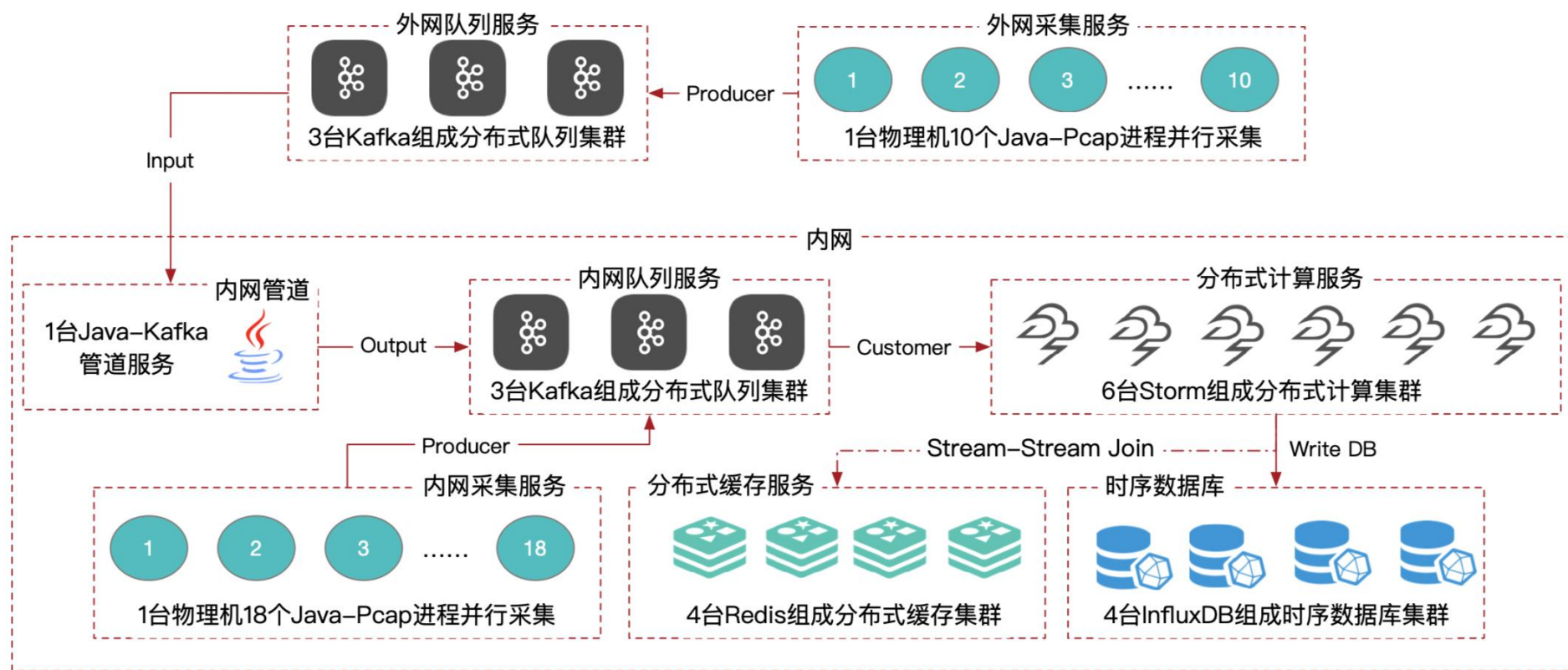
西北工业大学

NORTHWESTERN POLYTECHNICAL UNIVERSITY

1. 税务系统健康监测架构设计案例

● 税务系统健康监测数据处理部分的部署拓扑图

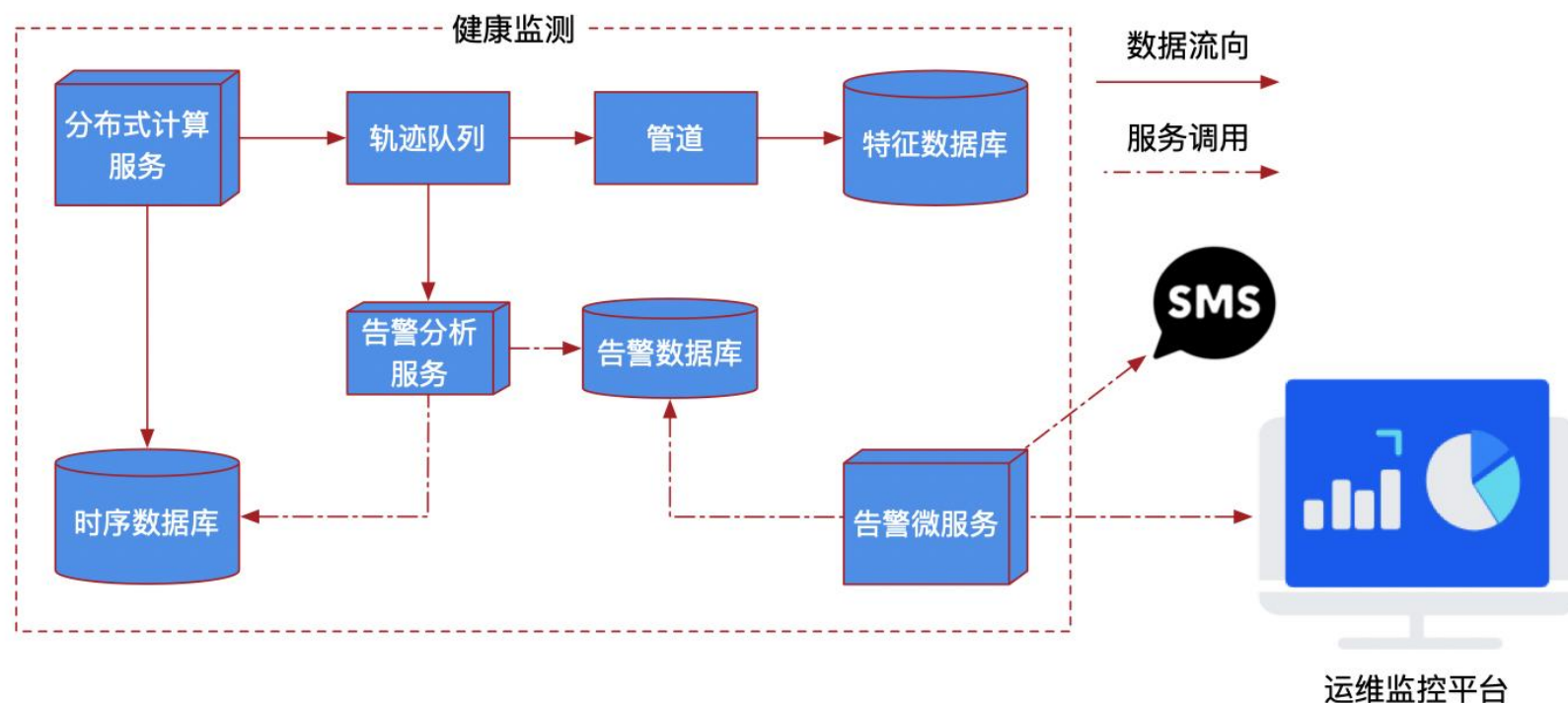
- 除采集服务为物理机外，虚拟化是支撑近**30**个集群节点的主要基础设施。
- 大数据系统架构的特点就是分布式协作、集群化服务。



1. 税务系统健康监测架构设计案例

● 税务系统健康监测的告警分析部分

- 告警分析服务通过对时序数据库进行聚合分析，判断规定时间窗口内的轨迹健康指标是否触发阈值。
- 告警微服务通过短信告警运维人员，并将异常定位信息与时间窗口通知运维监控平台。



1. 税务系统健康监测架构设计案例

- **InfluxDB**是一款专业的时序数据库（**TSDB**）

- **TSDB**数据结构：Point = 序列（Series）+ 指标项（Fields）+ 时间戳（Timestamp）

- **TSDB**的应用场景：

- 互联网平台的大量用户交互行为
 - 每天上亿的广告点击量实时分析
 - 交通系统的实时大数据处理
 - 网络通讯设备的数据流实时监控
 - 物联网传感器的运行指标监控

InfluxDB TSM数据模型在内存中的结构

Partition1	Map1	Series1+field1	Timestamp1 value1	Timestamp2 value2	
		Series1+field2	Timestamp1 value1	Timestamp2 value2	
			Timestamp1 value1	Timestamp2 value2	
.....					
Partition16	Map16	SeriesN+field1	Timestamp1 value1	Timestamp2 value2	
		SeriesN+field2	Timestamp1 value1	Timestamp2 value2	
			Timestamp1 value1	Timestamp2 value2	



1. 税务系统健康监测架构设计案例

● InfluxDB写入示例

○ 基于TCP数据包做吞吐量分析

- InfluxDB Point = 序列 (Series) + 指标项 (Fields) + 时间戳
- 服务器节点、源IP地址和端口、目标IP地址和端口组成序列 (Series)
- 增加两个指标项每秒数据包数量、每秒数据包的大小。

```
> SELECT count(*) FROM bps where time >= '2021-12-02T18:30:00+08:00' and time < '2021-12-02T18:31:00+08:00'
name: bps
time                count_num count_size
----                -
1638441000000000000 4480      4480
一分钟
每分钟4480条记录

> SELECT* FROM bps where time >= '2021-12-02T18:30:00+08:00' and time < '2021-12-02T18:31:00+08:00' order by time desc limit 10
name: bps
time                dst                node num size  src
----                -
1638441059000000000 172.17.120.163:39648 node4 1 229 172.17.120.160:8091
1638441059000000000 172.17.120.163:39648 node1 1 229 172.17.120.160:8091
1638441059000000000 172.17.120.163:22 node4 2 64 113.201.128.233:23749
1638441059000000000 172.17.120.162:59962 node3 253 789622 172.17.120.160:8088
1638441059000000000 172.17.120.162:59962 node1 249 789622 172.17.120.160:8088
1638441059000000000 172.17.120.162:52880 node3 7 308 172.17.120.160:8089
1638441059000000000 172.17.120.162:52880 node1 7 308 172.17.120.160:8089
1638441059000000000 172.17.120.162:52874 node3 14 616 172.17.120.160:8089
1638441059000000000 172.17.120.162:52874 node1 14 616 172.17.120.160:8089
1638441059000000000 172.17.120.162:44150 node3 2 40 172.17.120.161:2888
>
writeapi.writepoint(point),
}
```



1. 税务系统健康监测架构设计案例

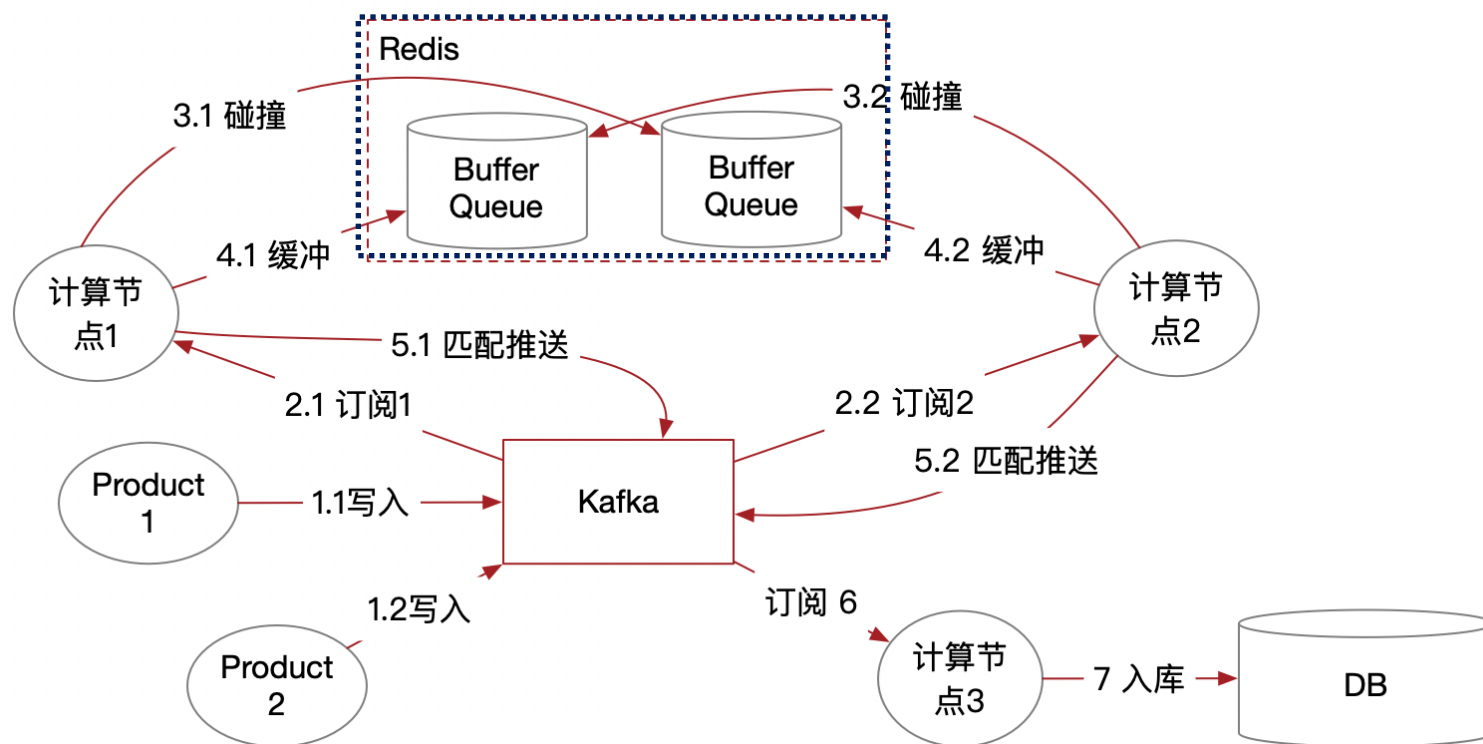
● 进一步理解Redis

○ Redis在流式架构中的作用

- 为了解决高速流数据临时缓冲并实时匹配计算的问题

两个数据节点在**Redis**最大堆积的量是**100万**条记录中进行实时匹配，匹配后立即被为清理。

若一个缓冲池的超出堆积总量或者某些记录超过存活时间，最早尚未匹配的过期记录也将被清理。

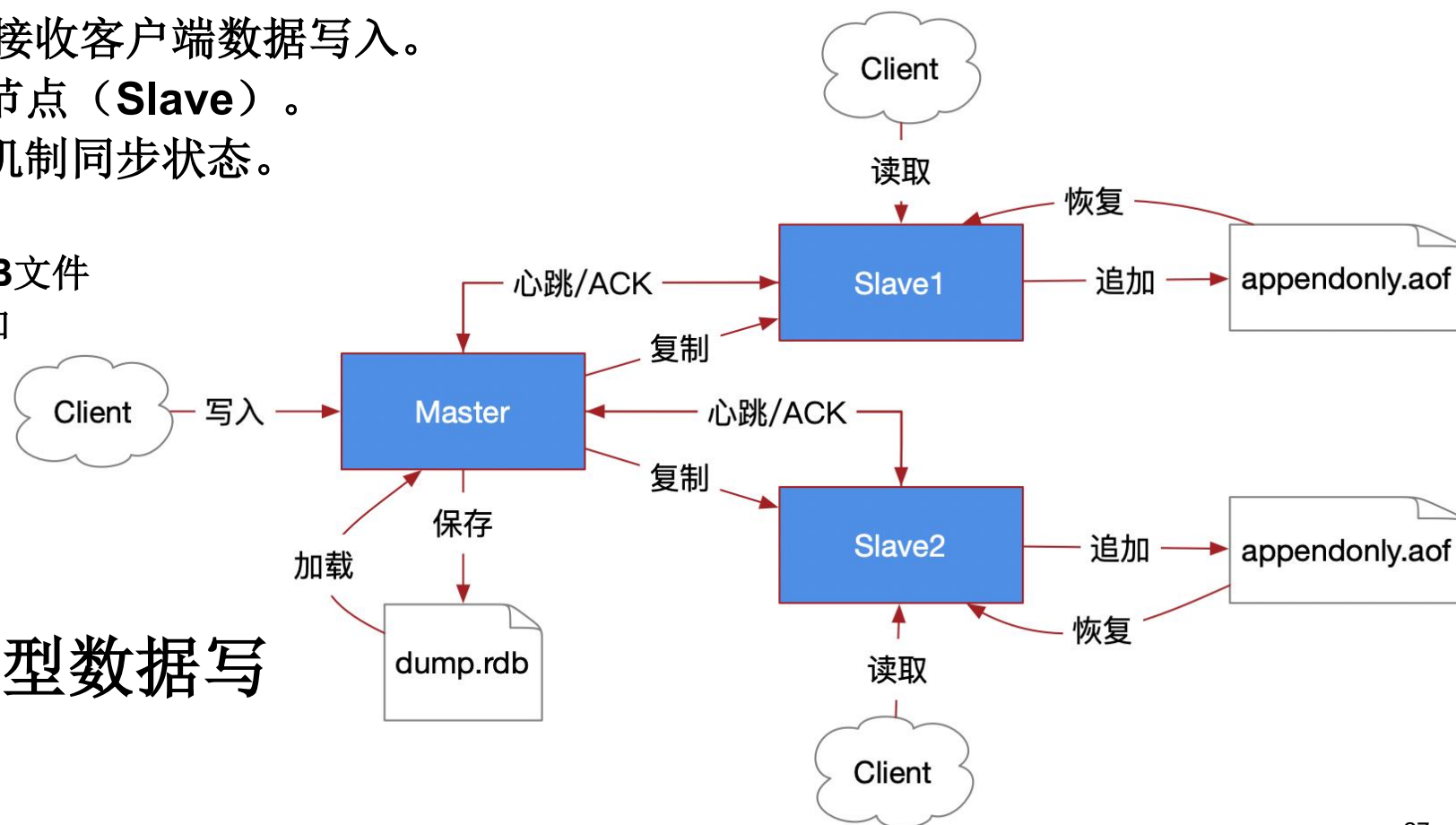


1. 税务系统健康监测架构设计案例

● 进一步理解Redis

○ Redis的主从复制架构

- 主节点（**Master**）接收客户端数据写入。
- 数据复制传播给从节点（**Slave**）。
- 主从节点通过心跳机制同步状态。
- 两种持久化方式：
 - 内存数据映射**RDB**文件
 - **AOF**文件日志追加



● 是否能够支撑密集型数据写入？

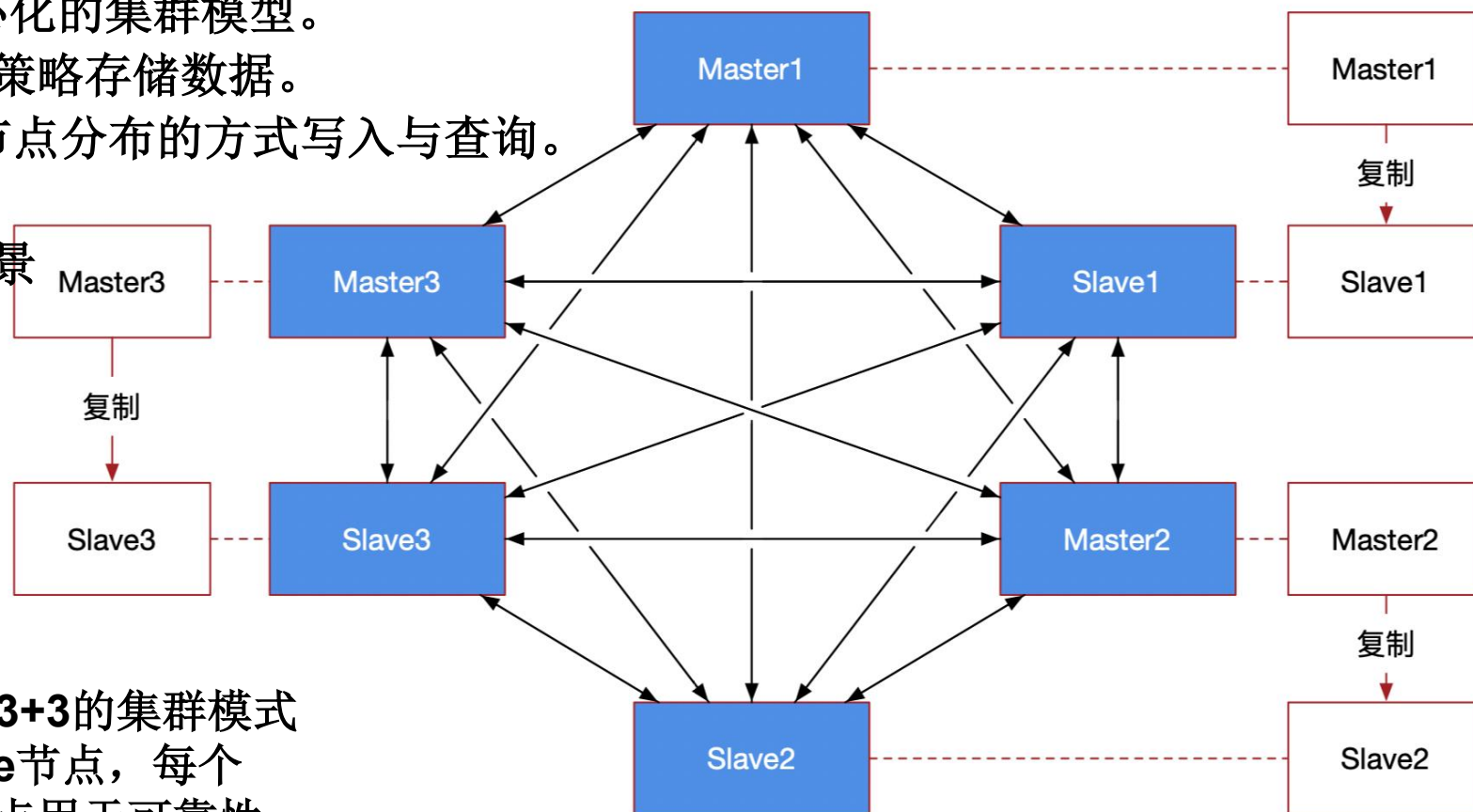


1. 税务系统健康监测架构设计案例

● 进一步理解Redis

○ Redis的集群架构

- Redis Cluster是去中心化的集群模型。
- 基于虚拟槽的哈希分区策略存储数据。
- 键(哈希值)->虚拟槽->节点分布的方式写入与查询。
- 适合密集型数据计算场景
- 适合抢票、秒杀等场景



图中采用了Redis Cluster3+3的集群模式
3个Master节点和3个Slave节点，每个
Master节点带1个Slave节点用于可靠性。



1. 税务系统健康监测架构设计案例

● 税务系统监测效果

- 当业务健康实时监测发现异常率过高时，定位相关服务资源，下钻平台监控，调取主机、数据库、中间件的实时监控数据进行分析。

内蒙国税-涉税业务健康度检测

实时 当日 本周 当月



内蒙国税-平台健康度检测



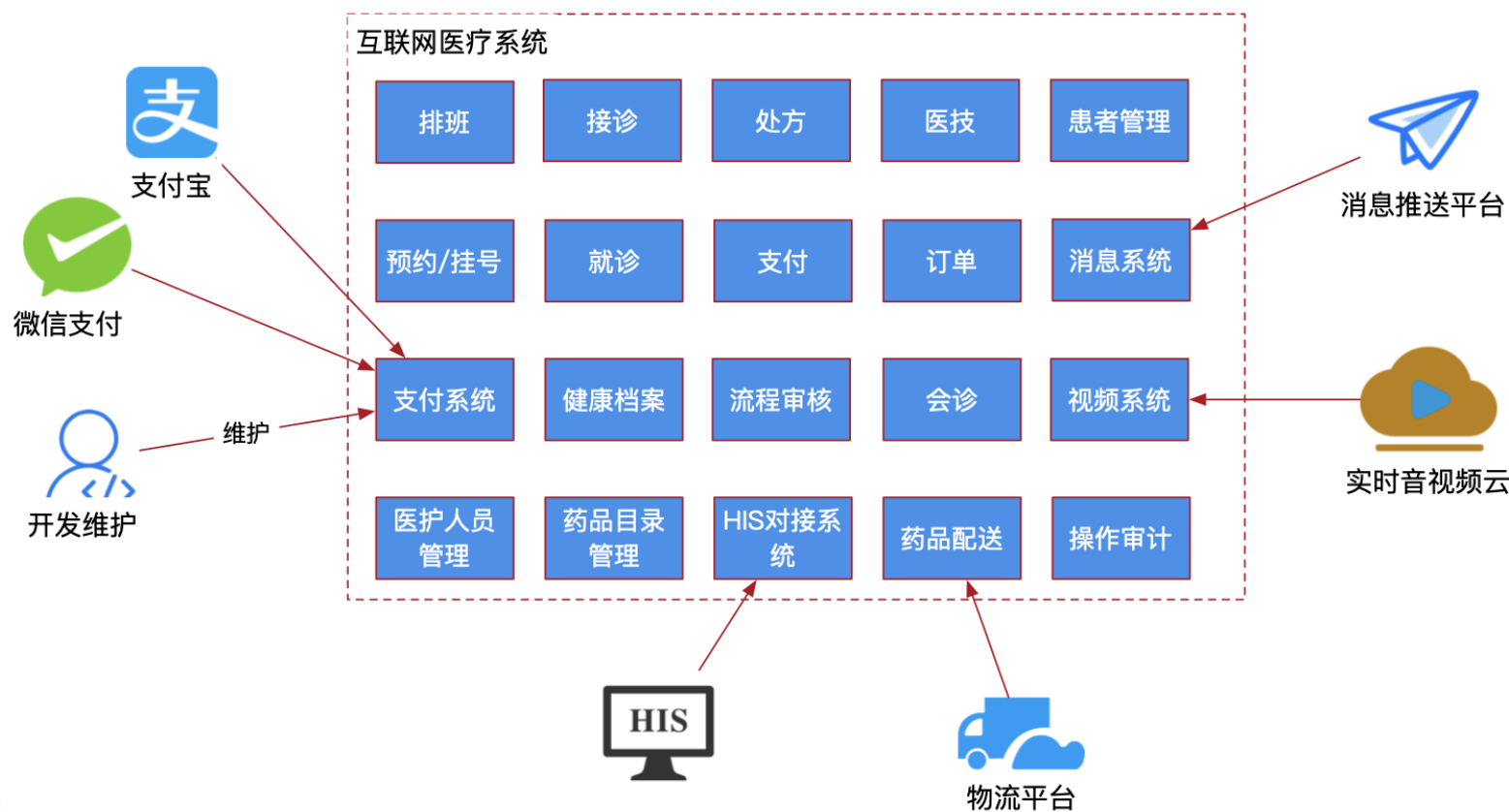
西北工业大学

NORTHWESTERN POLYTECHNICAL UNIVERSITY

2. 医疗系统数据同步架构设计案例

● 互联网医疗系统的业务架构

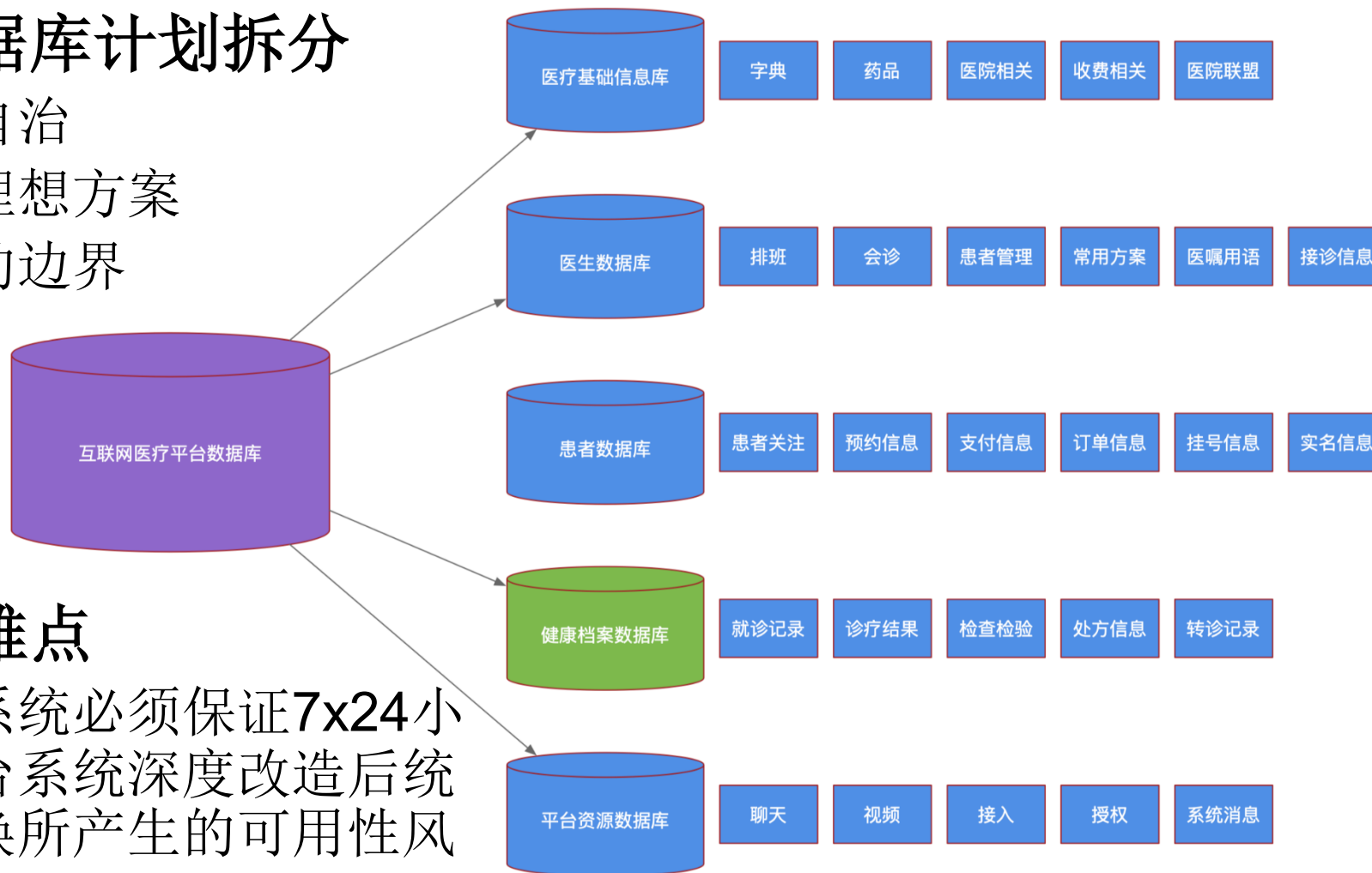
- 基于一个互联网平台，面向医院、医生、患者的在线医疗问诊。
- 需要对接大量外部互联网服务和院内HIS系统。



2. 医疗系统数据同步架构设计案例

● 互联网医疗平台数据库计划拆分

- 满足微服务的独立自治
- 解决高并发负载的理想方案
- 明晰了服务高可用的边界



● 拆分和数据迁移的难点

- 高可用问题：医疗系统必须保证7x24小时在线。但是大平台系统深度改造后统一部署、发布、替换所产生的可用性风险极大。

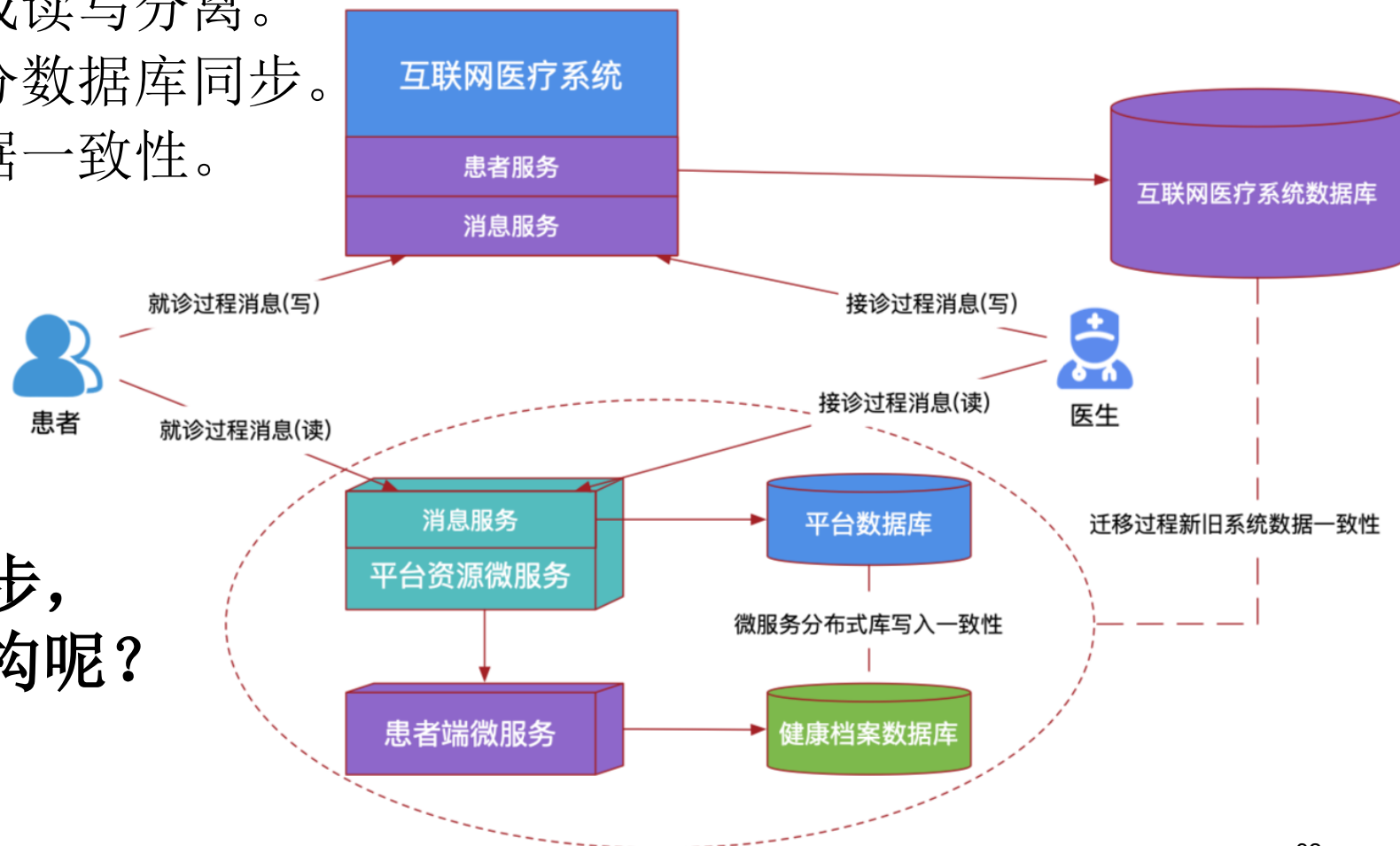


2. 医疗系统数据同步架构设计案例

- 数据库迁移过程中保持一致性

- 用户在新老数据库形成读写分离。
- 老数据库实时向新拆分数据库同步。
- 新数据库之间保持数据一致性。

- 完成实时在线数据同步，需要怎样设计系统架构呢？



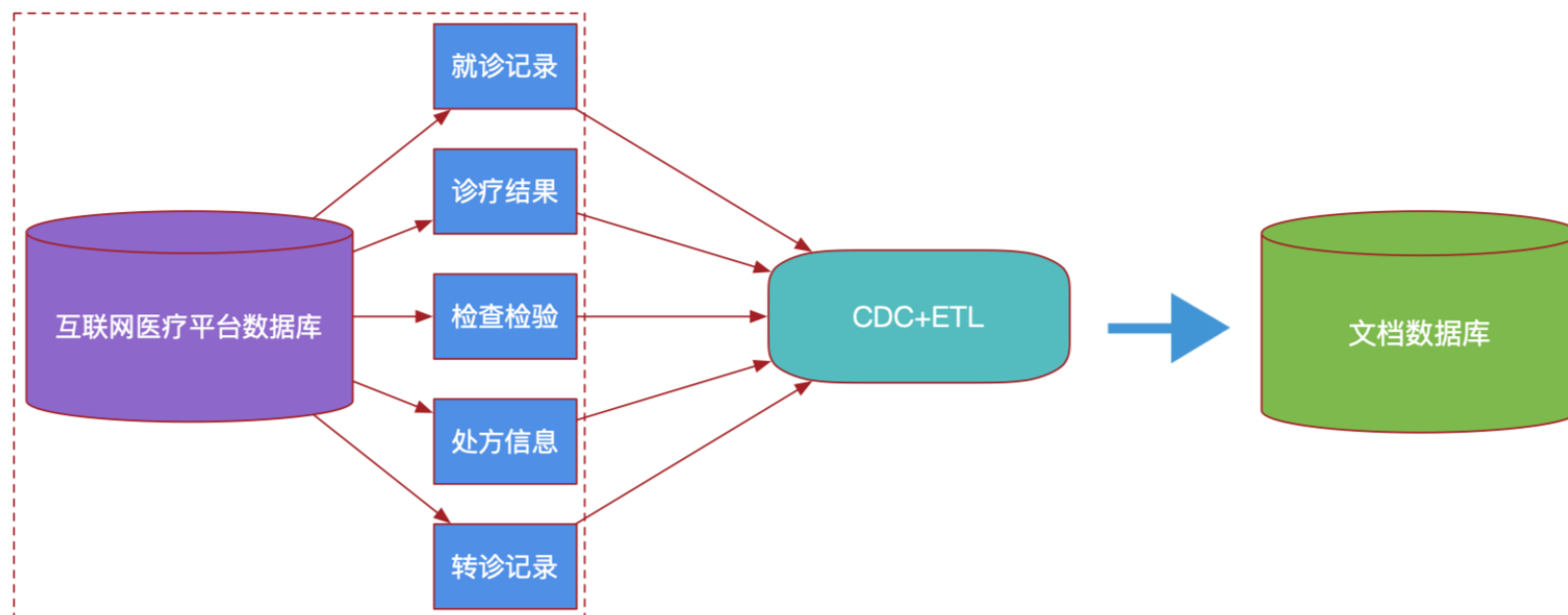
2. 医疗系统数据同步架构设计案例

● 数据平滑迁移的CDC+ETL同步架构

○ 健康档案数据迁移为例：

- 对数据库表数据变更捕获（CDC）方式采集。
- 通过ETL预处理，形成流式队列。
- 基于流库连接的方式，还原健康档案记录。
- 完成健康档案的文档数据库迁移。

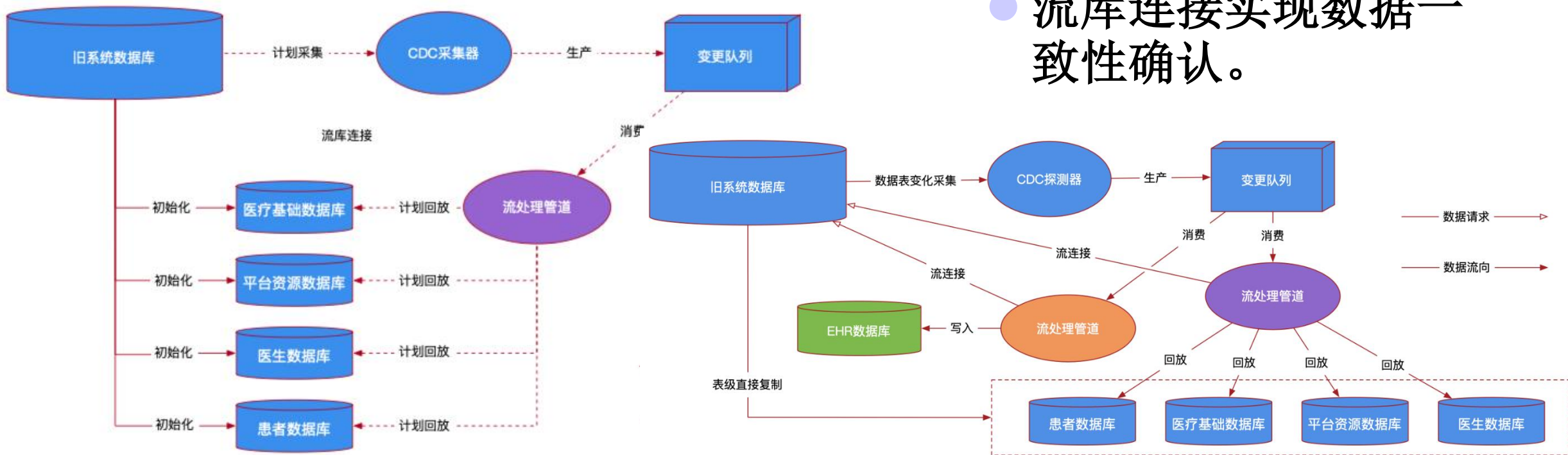
● 我们先看看ETL如何形成流式队列？



2.医疗系统数据同步架构设计案例

- ETL数据同步架构的步骤

- 步骤一：ETL全量初始化，建立流式管道，计划新库实时同步。
- 步骤二：流库连接，同步微服务数据、健康档案数据库。



- 流库连接实现数据一致性确认。

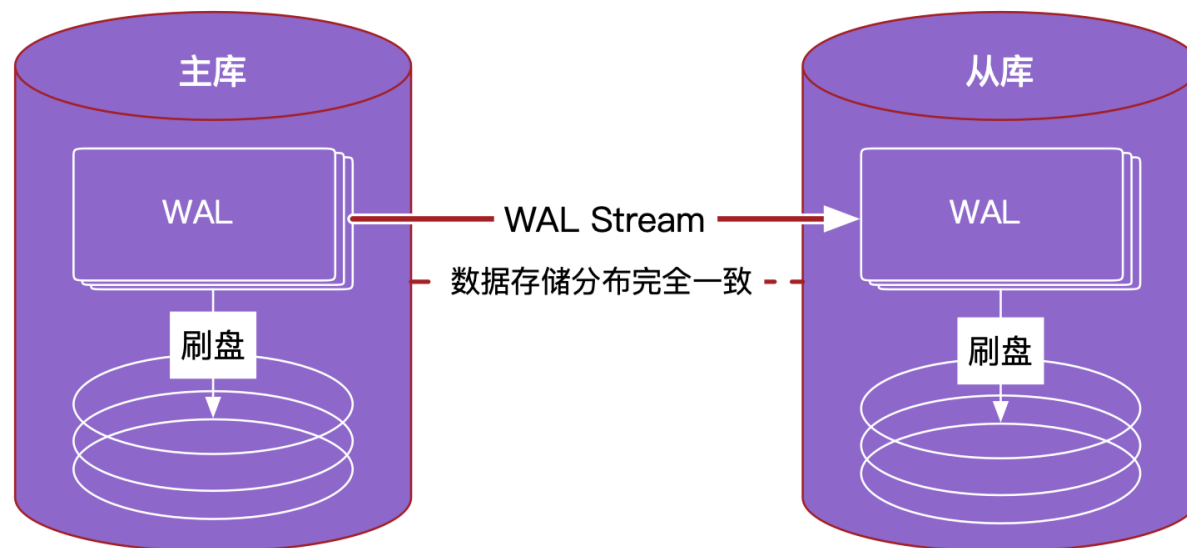


2. 医疗系统数据同步架构设计案例

- 医疗系统关系型数据库**CDC**（数据变更捕获）

- PostgreSQL流复制（也称为物理复制）。
- 通过WAL预写式日志，以WAL Stream的形式，将WAL RECORD传送到备用服务器实现块级别一致。

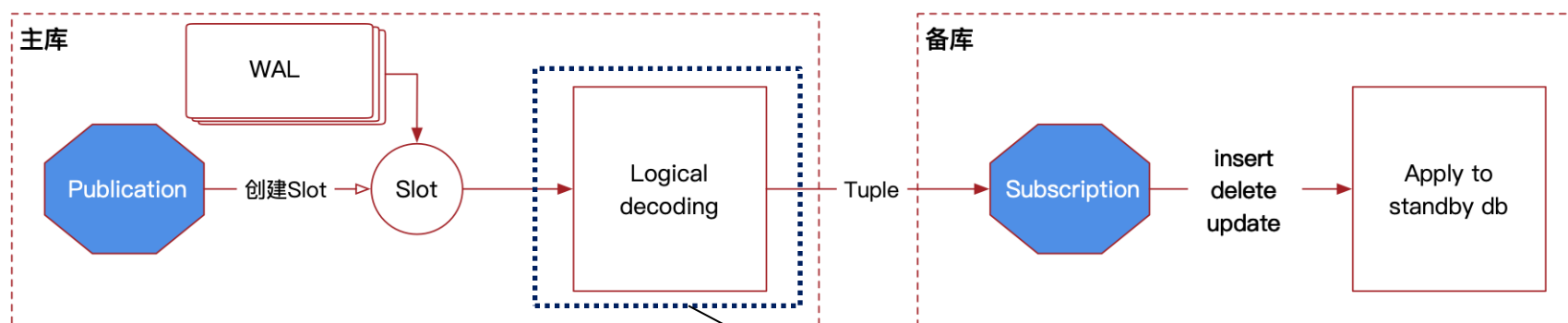
- 实时性最好，但没有介入的机会！



2. 医疗系统数据同步架构设计案例

- 医疗系统关系型数据库**CDC**（数据变更捕获）

- 逻辑复制不同于流复制，属于表级复制。
- 发布点**WAL**经过复制槽（**SLOT**）进行逻辑解码形成**Tuple**数据流。
- 订阅点获取**Tuple**数据流，转化为**SQL**语句回放。



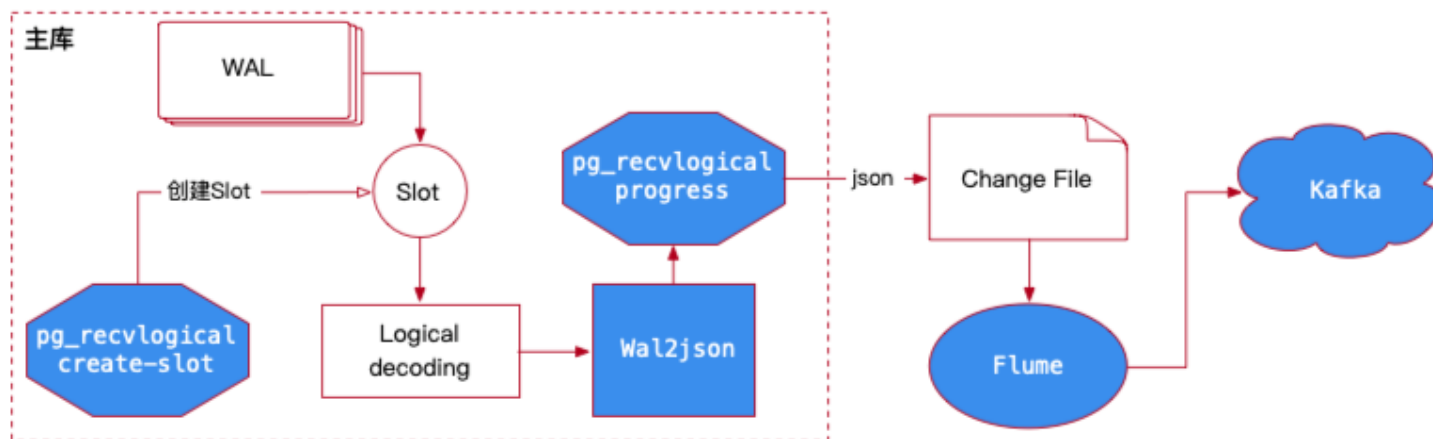
- **Logical decoding**机制提供了外部介入的可能！



2. 医疗系统数据同步架构设计案例

- 医疗系统关系型数据库**CDC**（数据变更捕获）

- 基于逻辑复制的WAL2JSON插件输出。
- WAL2JSON转换变更Tuple为JSON数据流，并写入日志文件。



- 变更记录进入文件后，就可以通过采集器例如**Flume**捕获记录并输出**Kafka**。



2. 医疗系统数据同步架构设计案例

● 医疗系统关系型数据库CDC（数据变更捕获）

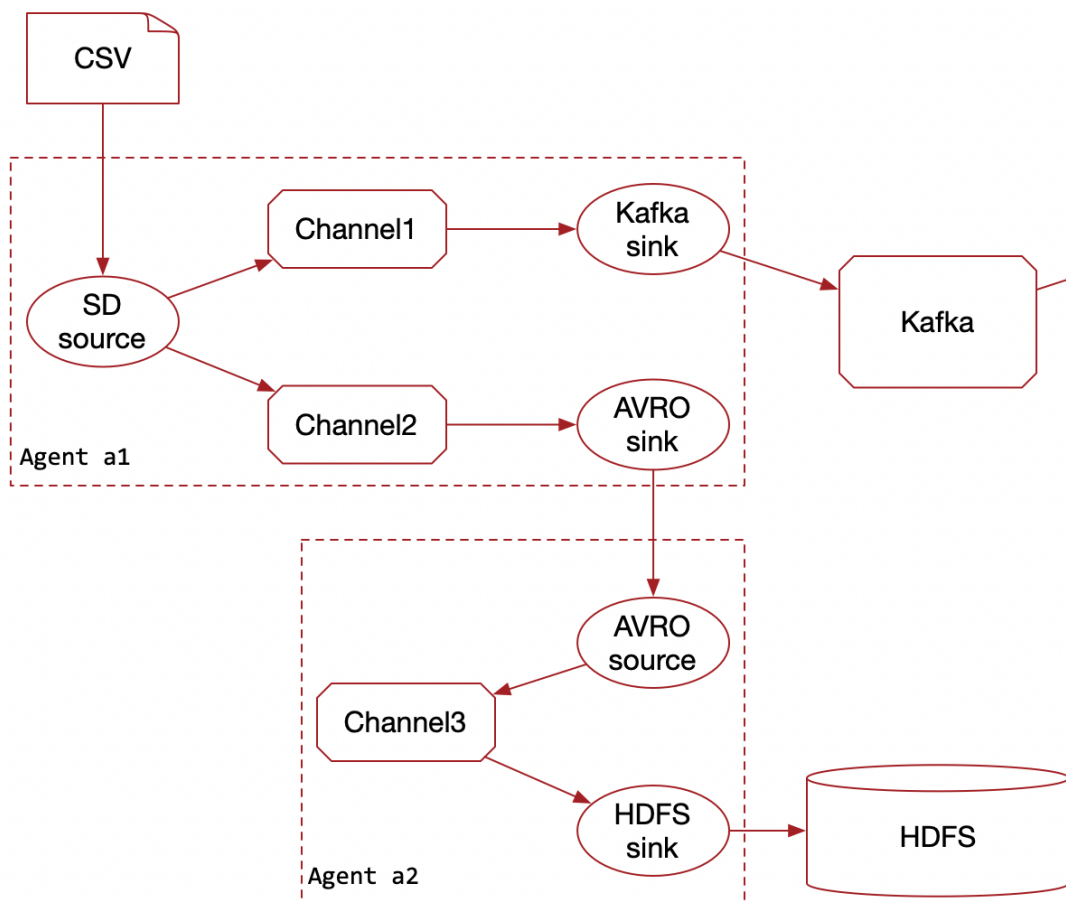
○ 数据演示

```
{
  "change": [
    {
      "kind": "delete",
      "schema": "health_master",
      "table": "treatment_result",
      "oldkeys": {
        "keynames": ["treatment_result_code"],
        "keytypes": ["character varying(50)"],
        "keyvalues": ["75bf401d-7ba4-465b-a7a2-791564d71e90"]
      }
    },
    {
      "kind": "insert",
      "schema": "health_master",
      "table": "treatment_result",
      "columnnames": ["treatment_result_code", "receive_code", "plan_code", "treatment_info", "patient_desc", "illness_desc", "treatment_result", "recommended_content", "doctor_code", "hos_code", "hos_area_code", "dept_code", "patient_code", "total_amount", "audit_status", "audit_status_name", "confirm_dept", "confirm_date", "confirm_user", "audit_dept", "audit_user", "status", "audit_date", "create_date", "create_user", "create_dept", "update_date", "update_dept", "update_user", "buss_init_type"],
      "columntypes": ["character varying(50)", "character varying(50)", "character varying(50)", "text", "character varying(500)", "character varying(500)", "character varying(500)", "character varying(500)", "character varying(50)", "character varying(50)", "character varying(50)", "double precision", "character varying(12)", "character varying(20)", "character varying(50)", "timestamp(6) without time zone", "character varying(50)", "character varying(50)", "character varying(50)", "character varying(1)", "timestamp(6) without time zone", "timestamp(6) without time zone", "character varying(50)", "character varying(50)", "timestamp(6) without time zone", "character varying(50)", "character varying(50)", "integer"],
      "columnvalues": ["75bf401d-7ba4-465b-a7a2-791564d71e90", "7a3d01fd-79cb-4f6d-9f50-bd6cec44607e", null, null, "糖尿病，一直在我院进行诊疗，现复诊开药。需服用'阿托他汀钙片-20mg'，'厄贝沙坦片-150mg'，药已用完。", "糖尿病，一直在我院进行诊疗，现复诊开药。需服用'阿托他汀钙片-20mg'，'厄贝沙坦片-150mg'，药已用完。", "糖尿病，高血压 动脉硬化", "继续药物治疗", "b12aab7b80c64149866aaa6383fd69c9", "20", null, "d817f257545440599f91b6e006dab1ad", "4b43e812-58c8-4e56-9975-6e8977f735cc", 24.71, "10", "待审批", null, null, null, null, null, null, "2020-05-21 15:00:28.224", "22944", "16", "2020-05-21 15:00:28.224", "16", "22944", 0]
    }
  ]
}
```

2. 医疗系统数据同步架构设计案例

- 医疗系统关系型数据库CDC（数据变更捕获）

- Apache Flume进行变更文件数据流采集。



train-1.conf: 扇形实例配置

配置Agent a1各个组件的名称

a1.sources = txt-source

#a1.sinks = hdfs-sink

a1.sinks = avro-sink kafka-sink

a1.channels = smem-channel-1 smem-channel-2

a1.sources.txt-source.type = spooldir

a1.sources.txt-source.spoolDir = /opt/flume-data

a1.sources.txt-source.fileHeader = true

#a1.sources.txt-source.inputCharset=UTF-8

a1.sources.txt-source.deserializer=LINE

指定 CSV 文件采集目录

a1.sinks.avro-sink.type = avro

a1.sinks.avro-sink.hostname = datanode-2

a1.sinks.avro-sink.port = 41414

Avro sink 指向 datanode-2 节点的 IP/端口

a1.sinks.kafka-sink.type = org.apache.flume.sink.kafka.KafkaSink

a1.sinks.kafka-sink.kafka.topic = flume-event-t1

a1.sinks.kafka-sink.kafka.bootstrap.servers = datanode-1:9092,datanode-2:9092,datanode-3:9092

a1.sinks.kafka-sink.kafka.flumeBatchSize = 20

a1.sinks.kafka-sink.kafka.producer.acks = 1

a1.sinks.kafka-sink.kafka.producer.linger.ms = 1

a1.sinks.kafka-sink.kafka.producer.compression.type = snappy

Kafka sink 指向 Kafka 集群

a1.channels.smem-channel-1.type = SPILLABLEMEMORY

a1.channels.smem-channel-1.memoryCapacity = 100000

a1.channels.smem-channel-1.overflowCapacity = 1000000

#a1.channels.smem-channel-1.byteCapacity = 800000

a1.channels.smem-channel-1.checkpointDir = /opt/flume/checkpoint

a1.channels.smem-channel-1.dataDirs = /opt/flume/tmp

a1.channels.smem-channel-2.type = SPILLABLEMEMORY

a1.channels.smem-channel-2.memoryCapacity = 100000

a1.channels.smem-channel-2.overflowCapacity = 1000000

#a1.channels.smem-channel-2.byteCapacity = 800000

a1.channels.smem-channel-2.checkpointDir = /opt/flume/checkpoint/2

a1.channels.smem-channel-2.dataDirs = /opt/flume/tmp/2

两个 Channel 各负责各自的 Sink

a1.sources.txt-source.channels = smem-channel-1 smem-channel-2

#a1.sinks.hdfs-sink.channel = smem-channel-1

a1.sinks.avro-sink.channel = smem-channel-1

a1.sinks.kafka-sink.channel = smem-channel-2

Source 和 Channel, Channel 和 Sink 的对应关系



西北工业大学

NORTHWESTERN POLYTECHNICAL UNIVERSITY

2. 医疗系统数据同步架构设计案例

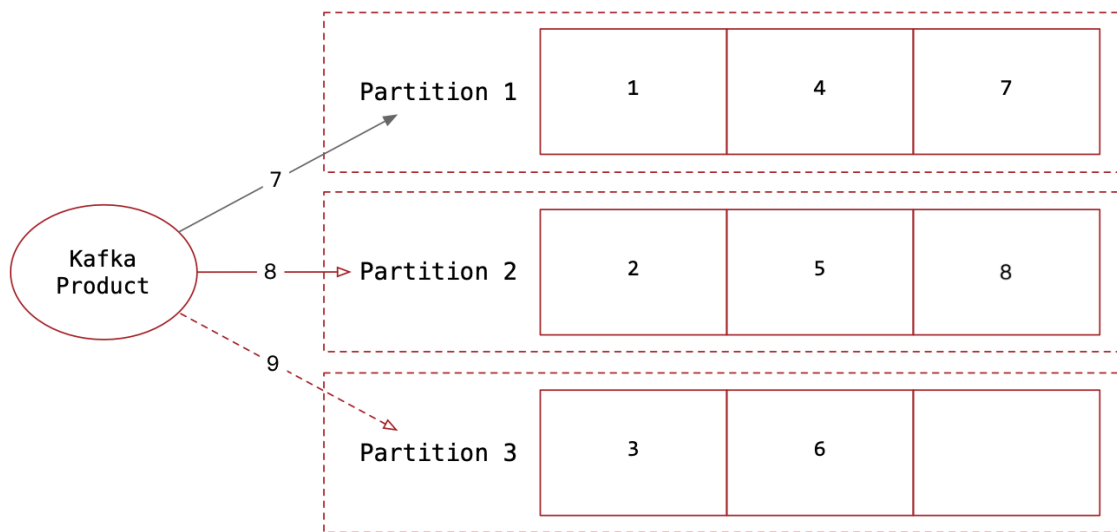
- 医疗系统关系型数据库CDC（数据变更捕获）

- Apache Kafka的分区策略。

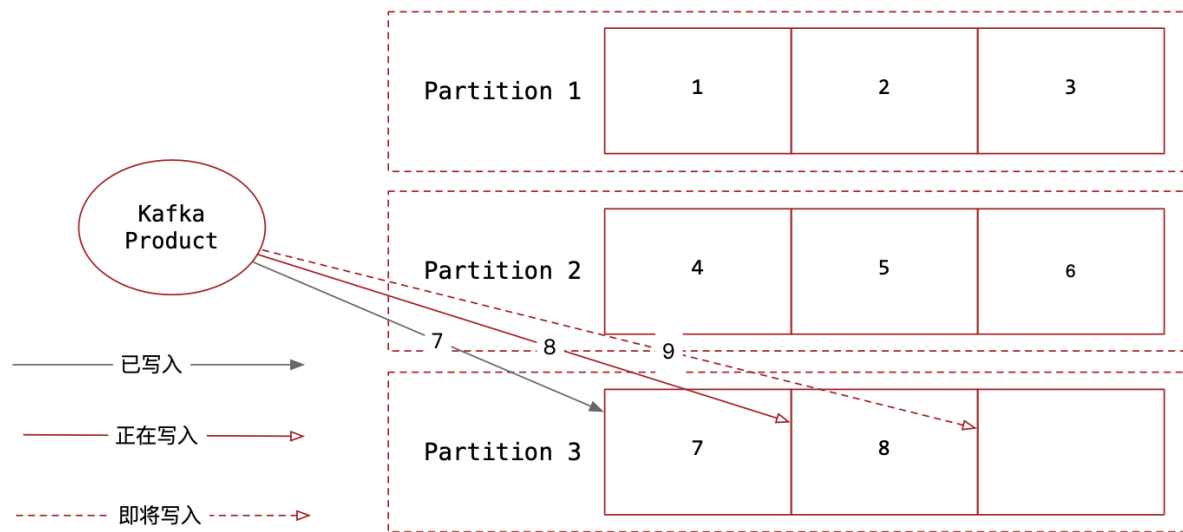
- Round-Robin分区策略
- Sticky分区策略

- 面向事务同步有什么问题？

Round-Robin partitioning strategy



Sticky partitioning strategy



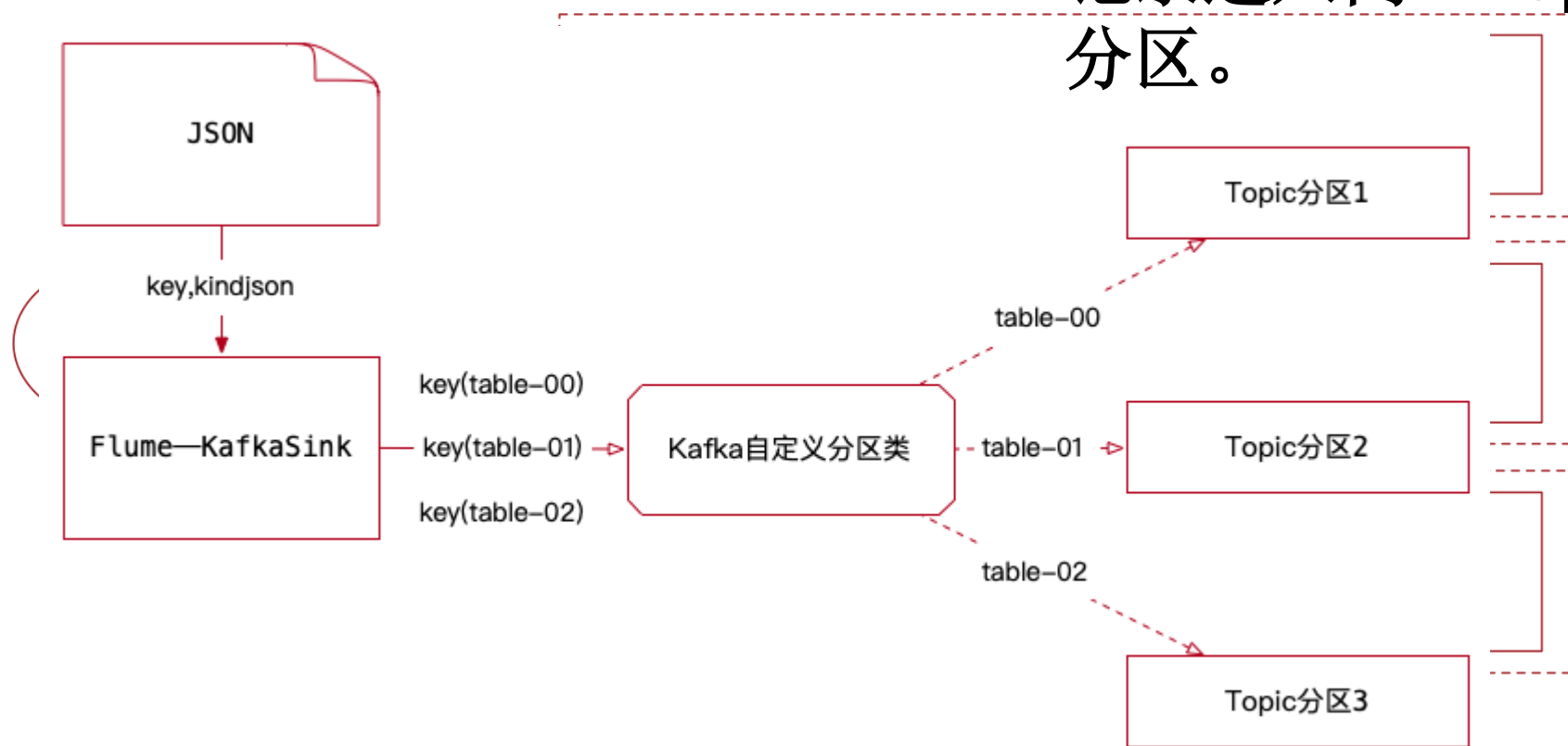
2. 医疗系统数据同步架构设计案例

- 医疗系统关系型数据库CDC（数据变更捕获）

- Apache Kafka的分区策略。

- 基于Key Hash的自定义分区。

- 让相同数据表的变更记录进入同一Topic分区。



2. 医疗系统数据同步架构设计

● 医疗变更记录的ETL回放

○ 构建健康档案（EHR）流库连接的实时



```
{
  "诊疗结果": {
    "患者": "",
    "医院": "",
    "医生": "",
    "处方明细": [
    ],
    "检查": [
    ],
    "检验": [
    ],
    "日期": "",
    "结果": "",
    "建议": ""
  }
}
```

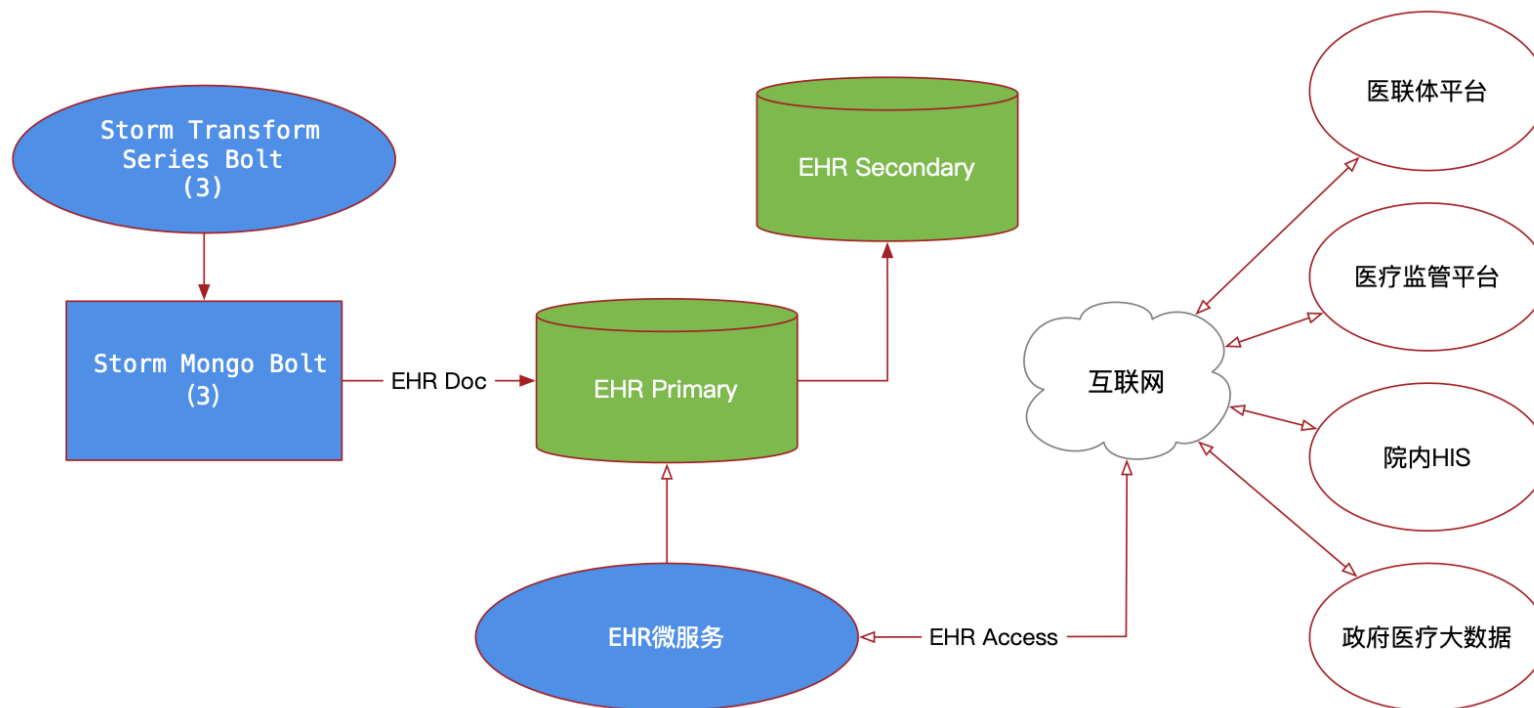
```
{
  "_id" : ObjectId("62356ff005b3540f85c352be"),
  "doctor" : "曹静",
  "result" : "按时用药, 定期复查",
  "doctorTitle" : "主任医师",
  "prescription" : [
    {
      "dosage" : "2",
      "no" : "2201191604162v11",
      "takingCycle" : "1",
      "orderNumber" : 1,
      "usage" : "静滴",
      "specification" : "2ml:20mg*1支/支",
      "measureUnit" : "ml",
      "packaging" : "支",
      "frequency" : "每天1次",
      "manufacturer" : "河南润弘制药股份有限公司",
      "unit" : "支",
      "instruction" : "1,5",
      "drugName" : "呋塞米注射液",
      "retailPrice" : "2.3000"
    }
  ],
  "advise" : "按时用药, 定期复查",
  "patient" : "曹静",
  "id" : "290d9ad4-4230-4a45-bb60-ca72ec80ae38",
  "office" : "内分泌代谢科",
  "hospital" : "延安大学附属医院",
  "type" : "在线复诊",
  "createDate" : NumberLong("1642578383810")
}

{
  "_id" : ObjectId("62356ff905b3540f85c352bf"),
  "doctor" : "曹静",
  "result" : "上呼吸道感染",
  "doctorTitle" : "副主任医师",
  "prescription" : [ ],
  "advise" : "口服药物治疗",
  "patient" : "曹静",
  "id" : "558blefc-5614-4b62-8609-333659ab6aa7",
  "office" : "肿瘤科",
  "hospital" : "陕西省森工医院",
  "type" : "在线复诊",
  "createDate" : NumberLong("1642981162130")
}
```



2. 医疗系统数据同步架构设计案例

- 基于MongoDB文档数据库的健康档案
 - 经过系列转换后形成的EHR Json文档写入MongoDB。
 - 通过EHR微服务建立EHR数据库与多种外部平台的访问连接。



2. 医疗系统数据同步架构设计案例

● 基于MongoDB文档数据库的健康档案

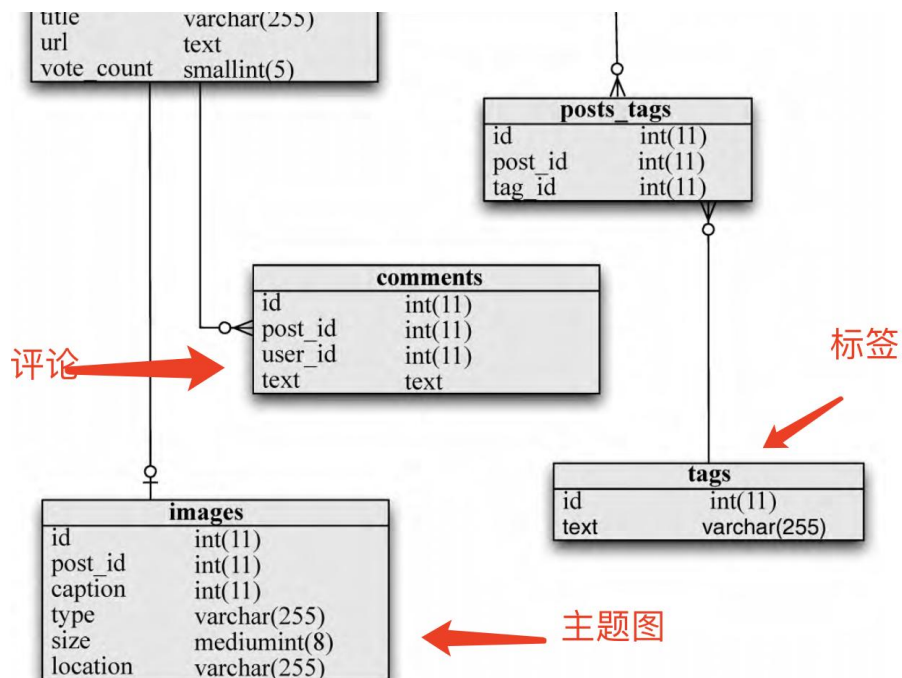
SQL查询带sports标签、投票数大于10的文章

```
SELECT * FROM posts
```

```
INNER JOIN posts_tags ON posts.id = posts_tags.post_id
```

```
INNER JOIN tags ON posts_tags.tag_id = tag.id
```

```
WHERE tags.text = 'sports' AND posts.vote_count > 10
```



```
{ _id: ObjectID('4bd9e8e17cefd644108961bb'),  
  title: 'Adventures in Databases',  
  url: 'http://example.com/databases.txt',  
  author: 'msmith',  
  vote_count: 20,  
  tags: ['databases', 'mongodb', 'indexing'],  
  image: {  
    url: 'http://example.com/db.jpg',  
    caption: '',  
    type: 'jpg',  
    size: 75381,  
    data: "Binary"  
  },  
  comments: [  
    { user: 'bjones',  
      text: 'Interesting article!'  
    },  
    { user: 'blogger',  
      text: 'Another related article is at http://example.com/db/db.txt'  
    }  
  ]  
}
```

← **_id**字段是主键

① 作为字符串数组存储的标签

② 指向另一文档的属性

③ 作为评论对象数组存储的评论

MongoDB中的等效查询，用文档做匹配

```
db.posts.find({'tags':'sports', 'vote_count': {'$gt':10}});
```



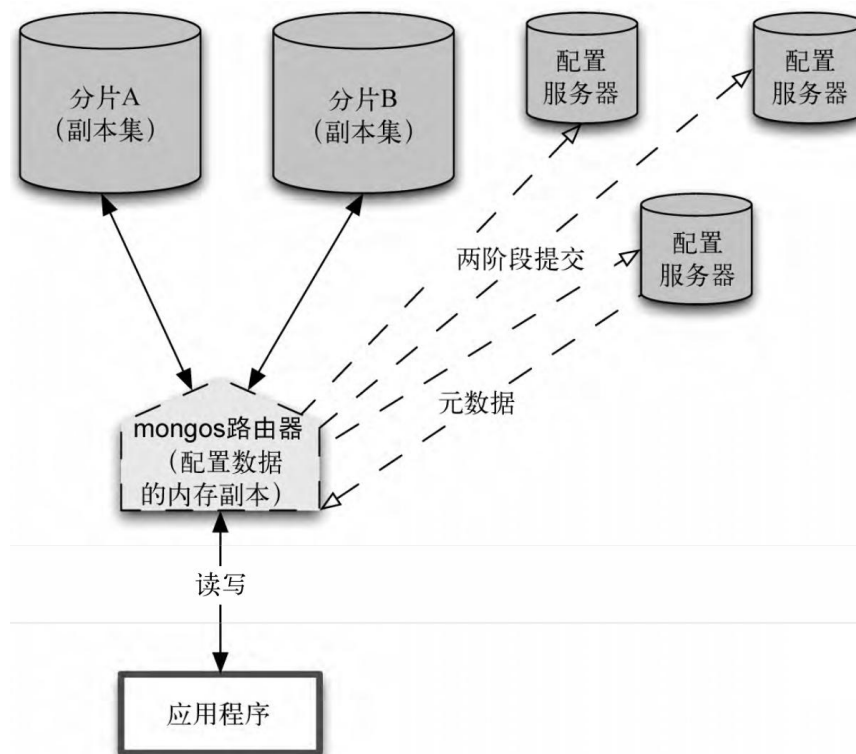
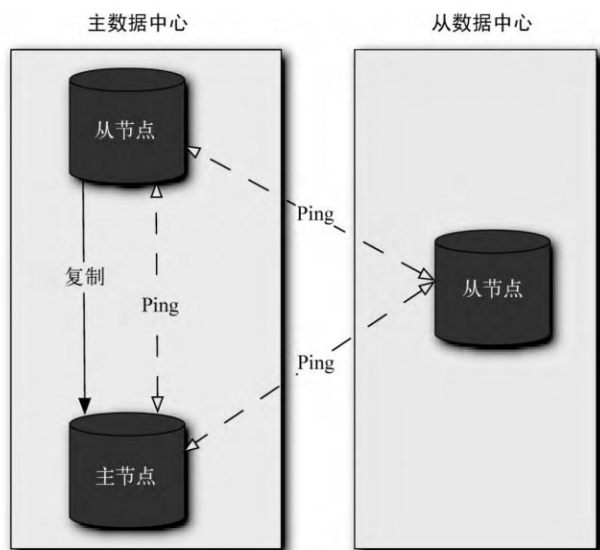
西北工业大学

NORTHWESTERN POLYTECHNICAL UNIVERSITY

2. 医疗系统数据同步架构设计案例

- 基于MongoDB文档数据库的健康档案

- MongoDB的副本集架构。
- MongoDB的水平扩展能力。

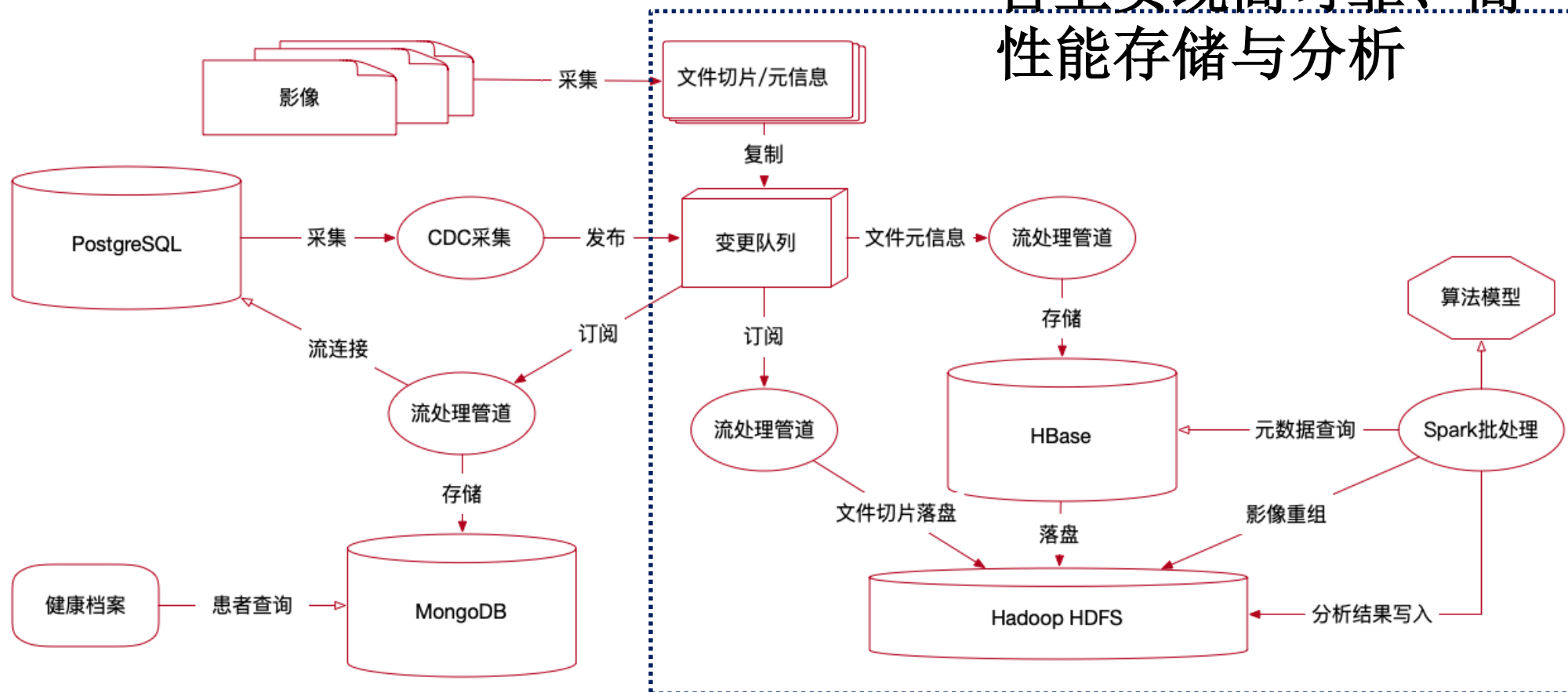


2. 医疗系统数据同步架构设计案例

- 影像文件数据同步与重组分析

- 健康档案同步、影像文件同步

- 影像文件在大数据平台上实现高可靠、高性能存储与分析



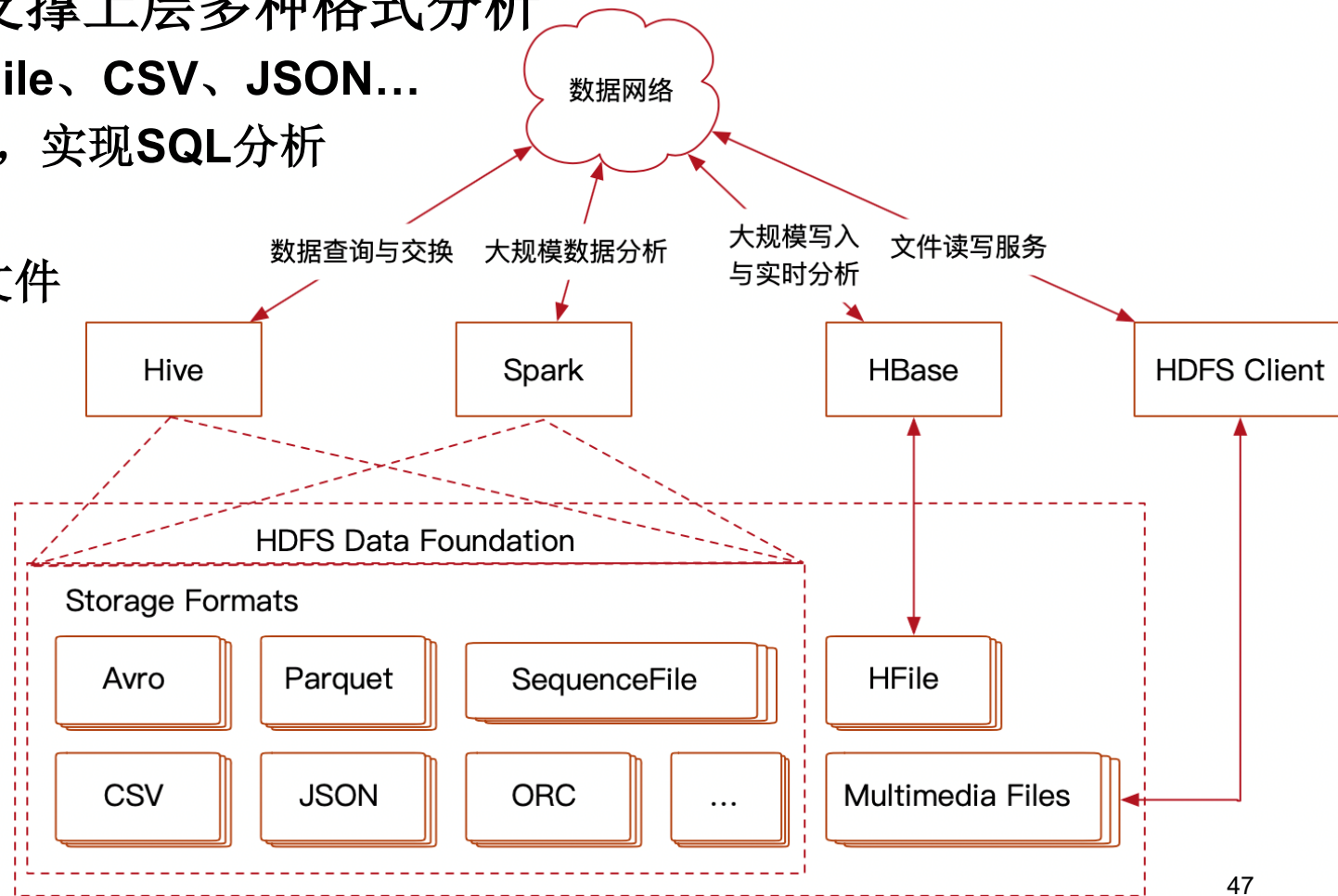
2. 医疗系统数据同步架构设计案例

● 大数据平台——Hadoop HDFS底座

○ HDFS数据底座可以存储和支撑上层多种格式分析

- Avro、Parquet、SequenceFile、CSV、JSON...
- Hive通过加载和写入这些文件，实现SQL分析
- Spark实现统一的RDD模型
- HBase会将数据落盘为HFile文件

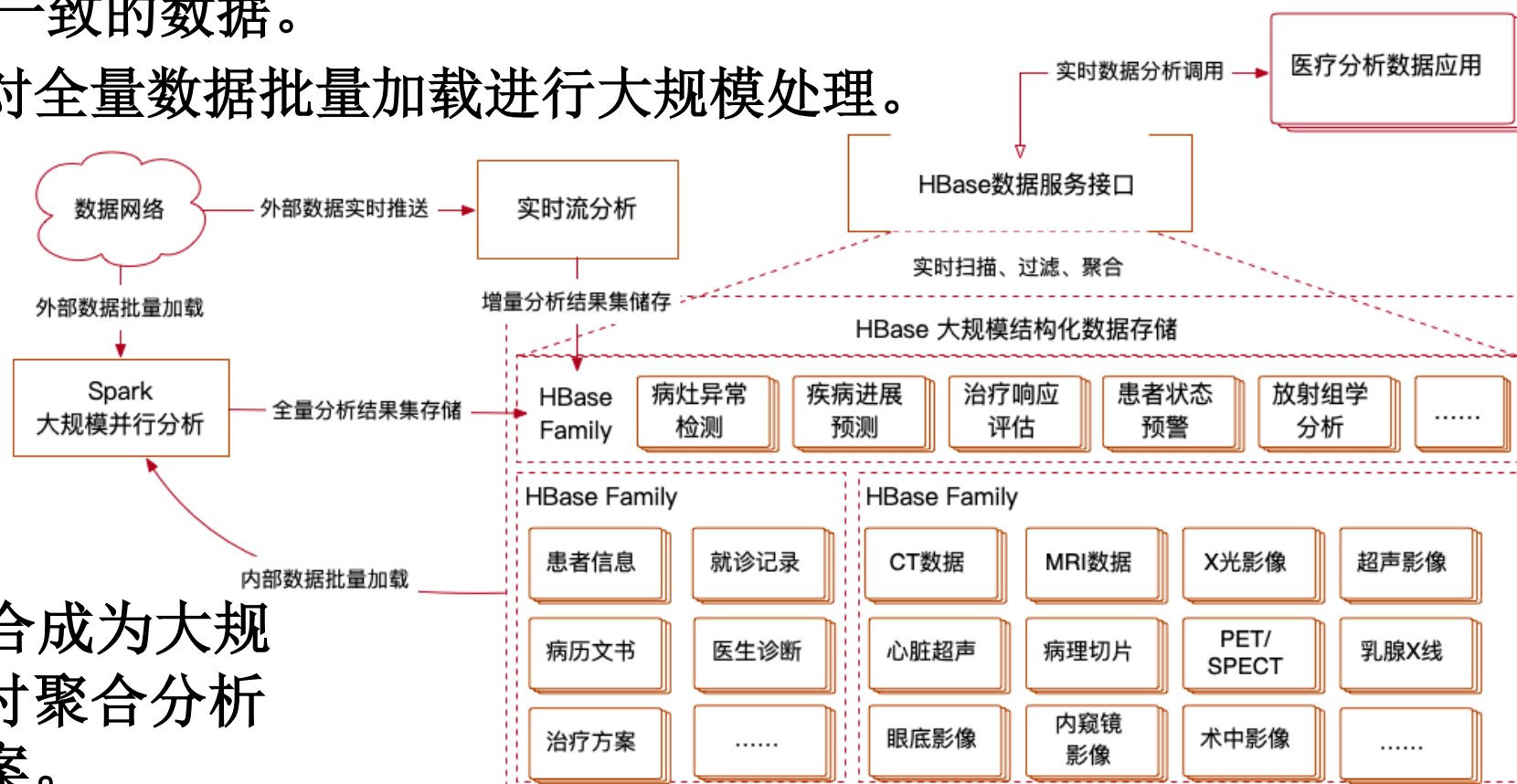
● Hadoop HDFS作为数据底座，形成了一个生态！



2. 医疗系统数据同步架构设计案例

● 大数据平台——HBase大规模结构化分析数据库

- HBase具备分布式能力，逻辑上非常宽的稀疏表，非常适合存储结构相似但字段又不完全一致的数据。
- Spark定时对全量数据批量加载进行大规模处理。



- HBase非常适合成为大规模数据进行实时聚合分析的数据存储方案。



2. 医疗系统数据同步架构设计案例

● 大数据平台——Spark大规模并行处理能力

○ Spark基础核心模型就是弹性分布式数据集（RDD）

- 医院每条产生上千张X光DICOM文件，多个文件形成一个HDFS数据块。
- 每晚Spark作业进行批量处理，加载数据块为RDD对象，进行X影像特征分析。
- 特征分析结果存储到HBase表，上层应用扫描表进行高风险分值聚合统计。

特征分析算法

